

PPML Is More Vulnerable to Cryptanalytic Extraction Attacks

Wen Zhang, Bingsheng Zhang*, Tianpei Lu, and Kui Ren

The State Key Laboratory of Blockchain and Data Security, Zhejiang University
{12421200, bingsheng, lutianpei, kuiren}@zju.edu.cn

Abstract. With the expansion of Machine Learning as a Service (MLaaS), Secure Multi-Party Computation (MPC) is widely used to protect the privacy of both proprietary models and client data during the inference process. To achieve practical performance, these protocols typically rely on fixed-point arithmetic over finite rings. However, this design choice introduces a unique arithmetic vulnerability: silent modular wraparound. In this paper, we propose a novel model extraction attack that actively exploits this behavior to accurately recover neural network parameters. Unlike existing methods that heavily rely on the non-differentiable points of piecewise linear activation functions (e.g., ReLU [CRYPTO 20, EUROCRYPT 25]), our attack leverages the discontinuous jumps triggered by modular wraparound. We successfully extract parameters from networks employing smooth activation functions (e.g., Swish, GELU) and effectively handle expansive network architectures where previous differential attacks fail. We present polynomial-time algorithms for recovering neuron signatures, norms, and signs, demonstrating that our approach remains highly robust even in restricted black-box scenarios where only top-1 label and probability are available to the attacker. Rigorous theoretical proofs and signal-to-interference ratio (SIR) analyses confirm that our sign recovery method significantly outperforms existing neuron wiggle techniques [EUROCRYPT24].

Keywords: Model Extraction Attack, Secure Multi-Party Computation, Modular Wraparound, Expansive Neural Network, Various Activation Functions

1 Introduction

With the popularization of Machine Learning as a Service (MLaaS), neural network models are being widely applied in sensitive fields such as medical diagnosis and financial risk control. However, sending data directly to the model owner or sending the model to the user poses severe privacy leakage risks. To address these privacy concerns, privacy-preserving machine learning (PPML) based on Secure Multi-Party Computation (MPC) has emerged. In typical MPC deployment scenarios, the data owner and the model owner collaboratively compute the neural

* Corresponding author

network’s inference results through cryptographic primitives like secret sharing, without revealing their private inputs (i.e., raw data and model parameters) to each other. Currently, a variety of advanced protocols and frameworks—such as SPDZ[9], SecureML[21], MiniONN[17], ABY3[20], SecureNN[27], CryptFlow2[23], Falcon[28], CryptGPU[25], SIRNN[22], and Piranha[29]—have been widely deployed to support such secure inference tasks. Under this architecture, computations are performed entirely on secret-shared values, guaranteeing that no intermediate values or parameters are exposed in plaintext.

Silent Modular Wraparound in MPC. Although MPC provides theoretical security, high computation and communication costs remain a major barrier to real-world deployment. While secure floating-point arithmetic exists, its high cost makes it impractical for running large deep neural networks. Therefore, most MPC frameworks use fixed-point arithmetic over finite rings (e.g., $\mathbb{Z}_M$ with $M = 2^{64}$) to approximate real-number operations. Specifically, a real number x is encoded into a ring element $\tilde{x} \in \mathbb{Z}_M$ using a scaling factor S , such that $\tilde{x} = \lfloor x \cdot 2^S \rfloor \pmod{M}$. While this design makes secure inference much faster, it introduces a subtle vulnerability: *silent modular wraparound*. In $\mathbb{Z}_M$, the canonical representation of integers lies in the range $[-M/2, M/2)$. However, when a computed value (such as a large vector dot product) falls outside this valid range, the MPC protocol does not raise an error or clip the value as a standard CPU would. Instead, it silently wraps around: the value is reduced modulo M into the canonical range, which is equivalent to adding or subtracting a non-zero integer multiple of M to the original value. This results in a sudden large jump in the numerical value, a discontinuous behavior that never occurs in standard plaintext neural networks.

Cryptanalytic Extraction Attacks. Cryptanalytic extraction attacks treat the target neural network as a black-box cryptographic oracle. By adaptively querying this black-box model and analyzing its outputs, attackers aim to reverse-engineer and precisely recover all the underlying parameters of the model. Existing state-of-the-art extraction attacks against neural networks [4,5,6,7,8] typically achieve this by constructing linear systems from input-output pairs, utilizing first-order derivative differences to formulate parameter equations [6,8], or leveraging the geometric property that the normal vector of a critical hyperplane is collinear with the neuron weights [5]. However, a fundamental bottleneck plagues all these techniques: they are structurally restricted by the specific architectural design of the target model. Specifically, current extraction methods face two significant challenges that render them largely ineffective against modern network topologies:

Challenge 1: Inability to Handle Smooth Activation Functions. Previous methods for recovering neuron weights highly rely on the non-differentiable point property of piecewise linear activation functions (e.g., the derivative of ReLU is discontinuous at 0: $\lim_{x \rightarrow 0^-} \sigma'(x) = 0$ and $\lim_{x \rightarrow 0^+} \sigma'(x) = 1$). This characteristic partitions the input space $\mathbb{R}^{d_0}$ into multiple polytopes; within each

polytope, the network behaves linearly, and the boundaries between polytopes are critical hyperplanes (i.e., hyperplanes satisfying $Wh + b = 0$). The attacker only needs to calculate the gradient difference on both sides of the critical hyperplane [6], or find the normal vector of this plane in the i -th layer’s input space [5], to recover the weight of the corresponding neuron. However, smooth activation functions (such as Swish, GELU, etc.) are C^∞ continuous. Therefore, such functions lack points of derivative discontinuity, making it impossible to locate the critical hyperplanes. This renders previous methods that rely on gradient mutations ineffective.

Challenge 2: Inability to Handle Expansive Networks. As indicated by Chen et al. [8], existing cryptanalytic attacks cannot recover neuron weights in expansive networks. Taking the extraction strategy proposed by Carlini et al. [6] as an example, this method computes the difference in directional derivatives on both sides of the critical hyperplane and selects d_{i-1} directions to solve for the weight w . However, for piecewise linear activation functions, near the critical point x^* , the computation of the previous $i - 1$ layers can be approximately expressed in the affine form of $\Gamma x + U$. Thus, the effective degrees of freedom for the i -th layer’s input near x^* are actually limited by the minimum output dimension of the first $i - 1$ layers, $\min_{k < i} \{d_k\}$. In expansive networks, the expansion of dimensions results in insufficient degrees of freedom to provide enough linearly independent vectors to uniquely determine the neuron weights.

Cryptanalytic Attacks on PPML. Prior work on PPML systems, MUSE[15], demonstrates that a malicious client can successfully extract model parameters during secure inference. However, this attack relies on a strong assumption: it requires the attacker to actively deviate from the cryptographic protocol to succeed. In contrast, our attack operates under a much weaker and more realistic threat model. We do not require any protocol violations; we only require the standard setting where the PPML system performs normal inference over finite rings.

1.1 Our Contributions

While MPC is fundamentally designed to protect privacy, we reveal a striking irony: the very arithmetic mechanisms meant to secure the model inadvertently create a backdoor. In this paper, we propose a novel cryptanalytic extraction attack that exploits this paradox—specifically, the inherent silent modular wraparound characteristic of MPC protocols. Table 1 compares our results with those of prior cryptanalytic extraction attacks. The main contributions of this paper are summarized as follows:

Extraction Across Smooth Activation Functions. We propose polynomial-time algorithms capable of recovering parameters from neural networks with any unbounded activation function, regardless of smoothness. Our analysis begins

by establishing an equivalent architecture that enables the analysis of modular wraparound in the real number domain. Unlike prior methods that rely on gradient mutations and are thus confined to piecewise linear functions, our approach leverages the discontinuous jumps caused by modular wraparound to enable parameter extraction even from networks with smooth activations.

Effective Recovery in Expansive Networks. We introduce a solution space aggregation method to address the dimensional limitations faced by previous differential attacks in expansive architectures. Specifically, we discover a relationship between the normal vectors and the neuron weights. By working with the visible subspace at each wraparound point, we can determine the homology of these points. We then aggregate the weight components from homologous wraparound points to reliably recover the full neuron weights. Notably, the normal vectors of critical hyperplanes share the same relationship with the weights. Therefore, our aggregation method can be directly applied to prior attacks to overcome their dimensional limitations.

Better Sign Recovery Algorithm. We propose a neuron sign recovery algorithm that uses the gradient divergence present at wraparound points. By comparing the magnitudes of directional derivatives along specific vectors, we can reliably determine the hidden signs. Theoretical proofs and signal-to-interference ratio (SIR) analyses show that our sign recovery approach outperforms the neuron wiggle method proposed by Canales et al. [4].

Generalization to Restricted Top-1 Score Scenario. Our attack is practical and generalizable. We extend our method to restricted black-box scenarios where the oracle only returns the top-1 label and its confidence probability. Moreover, our signature recovery algorithm is robust against output noise, such as small perturbations added to the probabilities. This ensures reliable extraction even when defenders try to hide the exact results.

Experiments on CIFAR10 Networks. Using our signature, norm, and sign recovery algorithms under the MPC scenario, we successfully extract all parameters from eight real DNNs. Each network has 986,634 parameters, consists of 4 hidden layers (256 neurons per layer), and is trained on CIFAR10 classification using eight different activation functions (including GELU, Swish, Mish, etc.).

1.2 Organization

The remainder of this work is organized as follows. Section 2 lays the groundwork by introducing the mechanics of MPC fixed-point arithmetic and defining the equivalent network configurations. A comprehensive overview of our extraction attack is presented in Section 3. The technical core of the attack is subsequently detailed across three sections: Section 4 explains the wraparound points search, Section 5 covers signature and norm recovery, and Section 6 focuses on sign recovery. Finally, Section 7 provides a thorough empirical evaluation, followed by concluding remarks in Section 8.

Table 1: Comparison of our attack against state-of-the-art cryptanalytic model extraction methods. Our method overcomes the limitations of smooth activation functions and expansive networks, and can be generalized to the top-1 score scenario.

Work	Activation Functions	Expansive Network		Required API
		Signature	Sign	
CRYPTO20 [6]	ReLU	×	Exponential	Raw outputs [†]
EUROCRYPT24 [4]	ReLU	×	Polynomial	Raw outputs
ASIACRYPT25 [8]	PReLU	×	×	Top-k scores
EUROCRYPT25 [5]	ReLU	×	Polynomial	Top-1 label
Ours	Unbounded Activations	✓	Polynomial (Better)	Top-1 score

[†] In fact, the raw outputs scenario can be generalized to the top-k scores scenario using the method of Chen et al. [8].

2 Preliminaries

2.1 Basic Definitions and Notations

Definition 1. Given a modulus M , the finite ring $\mathbb{Z}_M$ is defined as the set of integers in the interval $[-M/2, M/2)$. For any integer $x \in \mathbb{Z}$, the wraparound operator $\mathcal{W}_M : \mathbb{Z} \rightarrow \mathbb{Z}_M$ is defined as:

$$\mathcal{W}_M(x) = x - k \cdot M$$

where $k = \lfloor \frac{x}{M} + \frac{1}{2} \rfloor$ is called the wraparound coefficient. This operator maps any integer back to the principal value interval of the ring $\mathbb{Z}_M$.

Definition 2. Given a scaling factor S , for any real number $x \in \mathbb{R}$, its fixed-point representation $\tilde{x}$ on the ring $\mathbb{Z}_M$ is defined as:

$$\tilde{x} = \mathcal{W}_M(\lfloor x \cdot 2^S \rfloor)$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer.

For two ring elements $\tilde{x}, \tilde{y} \in \mathbb{Z}_M$, addition is defined as modular wraparound after ordinary addition: $\tilde{x} \oplus \tilde{y} = \mathcal{W}_M(\tilde{x} + \tilde{y})$. Fixed-point multiplication involves three steps: multiplication, truncation, and wraparound¹: $\tilde{x} \otimes \tilde{y} = \mathcal{W}_M\left(\left\lfloor \frac{\tilde{x} \cdot \tilde{y}}{2^S} \right\rfloor\right)$. If $\tilde{x} \oplus \tilde{y}$ or $\tilde{x} \otimes \tilde{y}$ exceeds the valid range of $[-M/2, M/2)$, the wraparound phenomenon will occur, corresponding to a wraparound coefficient $k \neq 0$.

¹ In actual protocols like SPDZ, the product $\tilde{x} \cdot \tilde{y}$ is first computed at a higher bit-width (e.g., 128 bits) to preserve full precision, then divided by the scaling factor 2^S to restore the fixed-point format, and finally mapped back to $\mathbb{Z}_M$ via $\mathcal{W}_M$.

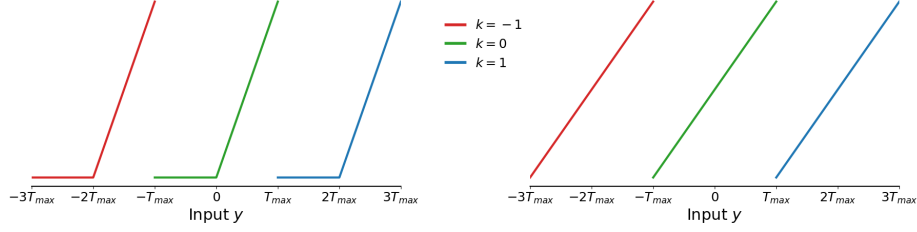


Fig. 1: The left panel shows the equivalent ReLU activation function and the right panel shows the input/output activation functions. The green segment ($k = 0$) represents the principal value interval without wraparound; the red segment ($k = -1$) and the blue segment ($k = 1$) represent the first wraparound intervals after negative underflow and positive overflow, respectively.

To extract parameters from a neural network that operates over a finite ring, we first construct a universal attack framework that is equivalent to the finite ring computation and can be rigorously analyzed in the real number domain.

Definition 3. An r -deep neural network $f_\theta : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_{r+1}}$ equivalent to the computation on a finite ring $\mathbb{Z}_M$ with a scaling factor S is defined as:

$$f = \sigma_{out} \circ f_{r+1} \circ \sigma \circ \dots \circ \sigma \circ f_2 \circ \sigma \circ f_1 \circ \sigma_{in}$$

where f_i denotes the linear transformation, σ represents the equivalent activation function, and σ_{in} and σ_{out} denote the input and output activation functions, respectively.

Definition 4. The i -th fully connected layer of a neural network is a function $f_i : \mathbb{R}^{d_{i-1}} \rightarrow \mathbb{R}^{d_i}$ given by the affine transformation:

$$f_i(x) = W^{(i)}x + b^{(i)}$$

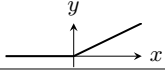
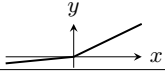
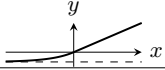
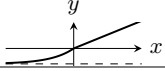
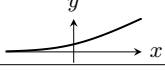
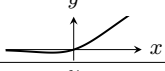
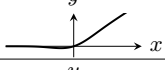
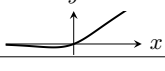
where $W^{(i)} \in \mathbb{R}^{d_i \times d_{i-1}}$ is the weight matrix, and $b^{(i)} \in \mathbb{R}^{d_i}$ is the bias vector.

Definition 5. The equivalent activation function σ is mathematically defined as:

$$\sigma(y) = \sigma_{original}(y - 2kT_{\max})$$

where $\sigma_{original}$ denotes the original continuous activation function (as summarized in Table 2). The parameter $T_{\max} = \frac{M}{2^{S+1}}$ represents the representational bound in the real domain, and $k \in \mathbb{Z}$ is the corresponding wraparound coefficient, uniquely determined by $k = \left\lfloor \frac{y}{2T_{\max}} + \frac{1}{2} \right\rfloor$. Furthermore, the specialized input and output activation functions are defined as: $\sigma_{in}(y) = \sigma_{out}(y) = y - 2kT_{\max}$. An illustrative plot of this periodic equivalent function is provided in Figure 1.

Table 2: Several commonly used unbounded activation functions, along with their analytical expressions and function plots.

Activation Function	Analytical Expression	Graph
ReLU	$\text{ReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$	
LeakyReLU	$\text{LReLU}(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases} \quad (\alpha \in (0, 1))$	
ELU	$\text{ELU}(x) = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases} \quad (\alpha > 0)$	
SELU	$\text{SELU}(x) = \lambda \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$	
Softplus	$\text{Softplus}(x) = \ln(1 + e^x)$	
Swish (SiLU)	$\text{Swish}(x) = \frac{x}{1 + e^{-x}}$	
GELU	$\text{GELU}(x) = \frac{1}{2}x \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right)$	
Mish	$\text{Mish}(x) = x \tanh(\ln(1 + e^x))$	

Definition 6. The j -th neuron in the i -th layer of the network is defined as:

$$\eta_j^{(i)}(x) = \sigma(W_j^{(i)}x + b_j^{(i)})$$

where $W_j^{(i)}$ denotes the j -th row vector of the weight matrix $W^{(i)}$. For an r -layer neural network, it contains a total of $\sum_{k=1}^{r+1} d_k$ neurons.

The key innovation of our equivalent model is to redefine the activation function σ , which allows us to analyze the problem in the real number domain. In this representation, modular wraparound in the MPC protocol is no longer seen as a computational anomaly. Instead, it becomes a periodic extension of the activation function. For the input and output layers, standard activation functions are not usually applied, but these layers are still affected by modular wraparound. Therefore, we introduce dedicated activation functions σ_{in} and σ_{out} for them. In the following analysis, we only consider inputs within the range $(-T_{\max}, T_{\max})^{d_0}$, so we omit the input activation function from the expressions.

Definition 7. The signature of a neuron $\eta_j^{(i)}$ is equal to $s_j^{(i)} \frac{W_j^{(i)}}{\|W_j^{(i)}\|_2}$, where $s_j^{(i)} \in \{-1, 1\}$.

Definition 8. The architecture of a fully connected neural network is uniquely described by its number of layers r , the dimension d_i of each layer where $i = 1, \dots, r+1$ and the choice of activation function σ .

Definition 9. The parameters θ of an r -layer deep neural network f_θ include the weights and biases of all layers in the network, i.e., $\theta = \{W^{(i)}, b^{(i)}\}_{i=1}^{r+1}$.

Let $F_i = \sigma \circ f_i \circ \dots \circ \sigma \circ f_1$ be the composite function of the first i layers, and $G_i = \sigma_{out} \circ f_{r+1} \circ \dots \circ \sigma \circ f_i$ the mapping from the i -th layer's input to the final output. Then $h^{(i)}(x) = F_i(x)$ is the feature vector for the $(i+1)$ -th layer, and $z^{(i)}(x) = W^{(i)}h^{(i-1)}(x) + b^{(i)}$ is the linear output before activation. Their j -th components are $h_j^{(i)}(x)$ and $z_j^{(i)}(x)$, respectively. For any $0 \leq i \leq r+1$, $f = G_{i+1} \circ F_i$. Here F_0 is the identity on the input space ($h^{(0)}(x) = x$), and G_{r+2} is the identity on the output space. See Figure 2 for an illustration.

2.2 Adversarial Goals and Assumptions

Our security model assumes the attacker can adaptively choose queries x as inputs to the oracle $\mathcal{O}_\theta$, and the oracle returns the corresponding predictions $\mathcal{O}_\theta(x)$. Based on a bounded number of $(x, \mathcal{O}_\theta(x))$ query pairs, the attacker aims to recover a set of parameters $\hat{\theta}$, with the goal of minimizing $\|\hat{\theta} - \theta\|$ as much as possible (up to permutation of neurons). We use $\hat{\cdot}$ to represent the recovered parameters.

Assumptions. We make the following assumptions regarding the oracle and the capabilities of the attacker:

- **Architecture knowledge.** We require exact knowledge of the model architecture of the neural network (e.g. depth, layer dimensions, activation function, and the representational bound $T_{\max}$).
- **Chosen inputs.** We can adaptively choose any input from $(-T_{\max}, T_{\max})^{d_0}$.
- **Precise computations.** The DNN is specified and evaluated using high-precision fixed-point arithmetic.
- **Unbounded activations.** The DNN employ any unbounded activation function.

3 Attack Overview

Because the activation function σ varies continuously in the interval $((2k-1)T_{\max}, (2k+1)T_{\max})$, $k \in \mathbb{Z}$, and is discontinuous at the points $(2k+1)T_{\max}$, $k \in \mathbb{Z}$, we call these points that cause discontinuous jumps in the output *wraparound points*. The following is the specific definition.

Definition 10. If for a certain input x , there exist $i, j, k \in \mathbb{Z}$ such that $z_j^{(i)}(x) = (2k+1)T_{\max}$, then x is called an *wraparound point of the neuron $\eta_j^{(i)}$* . We call *wraparound points originated from the same neuron homologous wraparound points*.

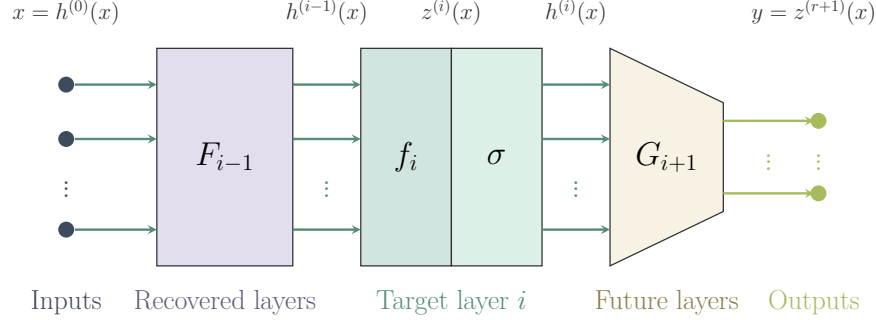


Fig. 2: Representation of the DNN as a composition of the recovered layers $1, \dots, i-1$, the current target layer i and the non-recovered future layers $i+1, \dots, r+1$.

Definition 11. A wraparound boundary of the k -th wrap corresponding to neuron $\eta_j^{(i)}$ is $B(\eta_j^{(i)}, k) = \{x | z_j^{(i)}(x) = (2k+1)T_{max}, x \in \mathbb{R}^{d_0}\}$. A signature plane corresponding to neuron $\eta_j^{(i)}$ at the k -th wrap is $S(\eta_j^{(i)}, k) = \{h | W_j^{(i)}h + b_j^{(i)} = (2k+1)T_{max}, h \in \mathbb{R}^{d_{i-1}}\}$.

We denote the local wraparound boundary around $x \in B(\eta_j^{(i)}, k)$ as $B_{x,R} = B(\eta_j^{(i)}, k) \cap N(x, R)$, where $N(x, R)$ represents the R -neighborhood of x . The normal vector of the wraparound boundary $B(\eta_j^{(i)}, k)$ at x is denoted as $n = \nabla_x z_j^{(i)}(x)$. These definitions can be referred to in Figure 3.

Similar to previous attacks, we view the neural network as a composite function consisting of three parts: the recovered layers F_{i-1} , the target layer i , and the non-recovered layers G_{i+1} . We adopt a layer-by-layer attack approach, meaning that when extracting the parameters of the i -th layer, we assume the parameters from the 1-st to the $(i-1)$ -th layer have been completely recovered. Figure 2 illustrates this view of the DNN.

For the parameter extraction of each layer, the process is divided into two core steps:

- **Signature and norm recovery:** This step aims to recover the signature and norm² of the weight vector and bias vector for each neuron, while their sign remains unknown, i.e., $W_j^{(i)} = \hat{W}_j^{(i)}$ or $-\hat{W}_j^{(i)}$.
- **Sign recovery:** This step aims to determine the sign of the neuron’s weight vector and bias vector.

² Conventional extraction attacks on piecewise linear activations recover only a neuron’s signature due to positive homogeneity. Arithmetic overflow disrupts this homogeneity, and our attack exploits this disruption to recover the exact norm of neurons.

Table 3: Impact of activation functions and network architectures on neuron signature recovery. The symbols $=$ and $\neq$ denote whether the affine span condition $\text{Aff}(B_{x,R}^{i-1}) = S(\eta_j^{(i)}, k)$ holds or fails, respectively.

Activation Function	Contractive	Expansive
ReLU	$\neq$ [6]	$\neq$ (Sec. 5.2)
LeakyReLU	$=$ [8]	$\neq$ (Sec. 5.2)
GELU, Swish, etc.	$=$ (Sec. 5.1)	$=$ (Sec. 5.1, 5.2)

The key difference between our attack and previous ones lies in the exploited feature. Previous attacks rely on the critical hyperplane $W_j^{(i)}h^{(i-1)} + b_j^{(i)} = 0$, whose normal vector $W_j^{(i)}$ is parallel to the neuron’s signature. However, such a hyperplane cannot be identified when the activation function is smooth. In contrast, our attack leverages the signature plane $S(\eta_j^{(i)}, k)$ of neuron $\eta_j^{(i)}$, which is parallel to $W_j^{(i)} \cdot h^{(i-1)} + b_j^{(i)} = 0$. The corresponding normal vector $W_j^{(i)}$ remains parallel to the neuron’s signature, and a wraparound boundary can be identified when an arithmetic wraparound occurs.

Therefore, obtaining the normal vector of the signature plane is sufficient to reconstruct the neuron’s signature. We denote $F_{i-1}(B_{x,R})$ as $B_{x,R}^{i-1}$. Since $B_{x,R}^{i-1} \subseteq S(\eta_j^{(i)}, k)$, it immediately follows that $\text{Aff}(B_{x,R}^{i-1}) \subseteq S(\eta_j^{(i)}, k)$, where Aff denotes the affine span. For contractive networks utilizing non-ReLU activations, the equality $\text{Aff}(B_{x,R}^{i-1}) = S(\eta_j^{(i)}, k)$ strictly holds, enabling the direct extraction of the normal vector from a single wraparound point³. However, both the expansive nature of certain architectures and the lossiness of ReLU can reduce the dimension of $B_{x,R}^{i-1}$ to strictly less than $d_{i-1} - 1$. Remarkably, we prove a non-trivial property regarding non-polynomial activation functions (e.g., GELU, Swish): their affine span remains the entire signature plane $\text{Aff}(B_{x,R}^{i-1}) = S(\eta_j^{(i)}, k)$ despite the dimensional deficiency $\dim(B_{x,R}^{i-1}) < d_{i-1} - 1$. These properties are detailed in Table 3.

As shown in Figure 3, it is a neural network based on GELU activation function. The local wraparound boundary $B_{x,R}$ is a one-dimensional curve segment in two-dimensional space, whereas $B_{x,R}^{i-1}$ is a one-dimensional curve segment in a three-dimensional space, which is contained within the two-dimensional signature plane $S(\eta_j^{(i)}, k)$. The one-dimensional curve segment can affine span the whole two-dimensional signature plane. Therefore, if $\text{Aff}(B_{x,R}^{i-1}) = S(\eta_j^{(i)}, k)$ holds, we can recover the neuron’s signature with a single wraparound point.

When $\text{Aff}(B_{x,R}^{i-1}) \neq S(\eta_j^{(i)}, k)$, recovering a neuron’s signature from a single wraparound point becomes mathematically infeasible. The attack proposed in [6] reconstructs signatures in contractive networks with ReLU activations by

³ A wraparound point is exploited along with its neighborhood.

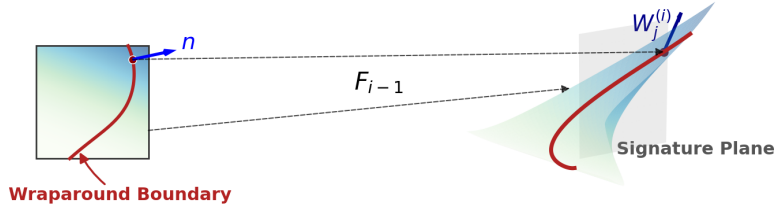


Fig. 3: This is a neural network based on the GELU activation function. F_{i-1} maps a local 2D green plane into a green curved surface in 3D space.

unifying partial solutions. Specifically, let r_1 denote the partial weight vector extracted at input x_1 , whose valid entries are located at a subset of indices $t_1 \subset \{1, \dots, d_{i-1}\}$ (respectively, r_2 at x_2 with index set t_2). Provided that x_1 and x_2 target the same neuron and their index sets overlap (i.e., $t_1 \cap t_2 \neq \emptyset$), the attacker can seamlessly merge these partial vectors to recover a broader weight vector spanning the union set $t_1 \cup t_2$.

However, in expansive networks, extracting valid entries at specific index subsets becomes unattainable. To overcome this, we propose a universal neuron signature aggregation method applicable to any activation function and expansive network. Instead of relying on the traditional method of recovering the signature via the difference matrix in the i -th layer's input space, our technique exploits the rigorous algebraic relationship $J(x)^T W^T = n$, which links the normal vector n at the wraparound boundary x to the target neuron's weight. Here, $J(x)$ denotes the Jacobian matrix as defined in Definition 12. This paradigm shift is motivated by our key observation: compared to relying solely on the difference matrix, this relationship inherently encapsulates the projection information of $W_j^{(i)}$, effectively integrating the structural dynamics of the preceding layers F_{i-1} into the extraction process.

Specifically, suppose x_a and x_b are two distinct wraparound points triggered by the same target neuron η . Let w_a denote the orthogonal projections of its signature onto the J_a subspace, (respectively w_b for J_b). Provided that the $J_a \cap J_b$ is non-empty, we can successfully aggregate these components to reconstruct the signature's projection onto the expanded space $\text{span}(J_a \cup J_b)$. Ultimately, if this combined subspace spans the entire feature space, the exact signature of the neuron can be uniquely determined.

Having extracted the signature and norm of the neuron, it remains necessary to determine its exact sign. Prior works have traditionally exploited the discontinuous derivatives of ReLU and PReLU at the origin. Specifically, Canales et al.[4] proposed a polynomial-time *neuron wiggle* technique applicable to ex-

pansive networks. Their approach leverages the observation that the network’s output exhibits a significantly higher rate of change as the input x transitions from the inactive region to the active region of the ReLU function. Unfortunately, this method is not directly applicable to smooth activation functions because of their everywhere-continuous derivatives.

We leverage the fact that the derivatives of all known unbounded activation functions are discontinuous at $(2k + 1)T_{\max}$ for any integer k , satisfying the inequality $\sigma'((2k + 1)T_{\max} - \epsilon) \gg \sigma'((2k + 1)T_{\max} + \epsilon)$. This property thereby leads to a directional derivative whose magnitude is greater on the active side of the boundary than on its inactive side. We model this directional derivative as a superposition of signals: a target signal emanating from the targeted neuron, and interference signals originating from the remaining neurons. On the active side, the derivative is primarily governed by the target signal, while on the inactive side, it reduces to interference-dominated signals. By observing a sufficient discrepancy in the magnitude of directional derivative across the two sides of this boundary, the sign of the neuron can be deduced. Therefore, we can elegantly reduce the sign recovery challenge to an optimization problem: maximizing the discriminability between the target signal and the interference signals. While the existing neuron wiggle method attempts to amplify the target signal, we demonstrate that its perturbation direction remains inherently sub-optimal. By constructing a direction that maximizes this discriminability of the signals, we recover the neuron’s sign with higher accuracy and lower query complexity than the neuron wiggle approach.

4 Wraparound Points Search

Our method for searching wraparound points relies on observing changes in the network’s output. As established in Theorem 1, the output changes continuously as long as the function does not cross a wraparound point. In contrast, Theorem 2 states that crossing a wraparound point results in a discontinuous change. The ReLU activation function exhibits the highest probability of continuous behavior, primarily because its parameter $p = 1/2$ makes it more susceptible to the non-activation region. Nevertheless, the overall continuity probability, given by $\sum_{k=i+1}^r p^{d_k}$, remains extremely small. Therefore, it is highly likely that crossing a wraparound point will induce a discontinuous and abrupt change. Consequently, we can detect the presence of a wraparound point by monitoring whether the output change is continuous. The detailed procedure is presented in Algorithm 5.

Theorem 1. *Assume no wraparound points exist in the ϵ -neighborhood of x . Let the norm of each neuron’s weight vector in layer i be bounded by K_i , i.e., $\|W_j^{(i)}\|_2 \leq K_i$ for all j , and let σ be a λ -Lipschitz continuous activation function. Then, for any perturbation with $\|\Delta x\|_2 < \epsilon$, the network output satisfies $\|f(x + \Delta x) - f(x)\|_2 \leq C\|\Delta x\|_2$, where $C = \lambda^r \prod_{i=1}^{r+1} \sqrt{d_i} K_i$.*

Theorem 2. Suppose x is a wraparound point in the i -th layer. Let $\mathcal{S}_\sigma = \{z \in \mathbb{R} \mid \exists \epsilon > 0, \forall y \in (z - \epsilon, z + \epsilon), \sigma(y) = \sigma(z)\}$ denote the flat region of the activation function, and let $p = \Pr[z \in \mathcal{S}_\sigma]$ be the probability that an intermediate pre-activation falls into this region. When traversing across this wraparound point, the probability that the network’s final output remains continuous is bounded by $\Pr[\lim_{\|\Delta x\| \rightarrow 0} f(x + \Delta x) = f(x - \Delta x)] < \sum_{k=i+1}^r p^{d_k}$.

Calculating the normal vector. Once a wraparound point x is precisely located, we compute the normal vector n of its associated wraparound boundary. Formally, n satisfies the following property: for any sufficiently small perturbation v , the point $x + (v - \text{proj}_n(v))$ lies on the tangent hyperplane whenever x does. Equivalently, displacements orthogonal to n do not leave the tangent hyperplane. Leveraging this geometric invariance, Algorithm 6 reconstructs n exactly through a sequence of oracle queries.

Remark. In the top-1 score scenario, the method for searching wraparound points remains largely unchanged from before. Rather than focusing on changes in the top-1 label, which may merely reflect normal class switches induced by decision boundaries, we should instead closely monitor discontinuous jumps in the confidence probability.

5 Signature and Norm Recovery

This section is divided into two parts: first, extracting the neuron signature $\hat{W}_j^{(i)} = c_j^{(i)} W_j^{(i)}$ containing an unknown scaling factor; then calculating the absolute value of this scaling factor $|c_j^{(i)}|$ (i.e., reciprocal of the norm).

5.1 Signature Recovery from a Single Wraparound Point

Definition 12. The Jacobian matrix associated with the i -th layer is defined as the matrix of partial derivatives of the post-activation features with respect to the network’s input, denoted as $J^{(i)}(x) = \frac{\partial h^{(i)}(x)}{\partial x}$.

The signature of a neuron $\eta_j^{(i)}$ is parallel to the normal vector of its corresponding signature plane $S(\eta_j^{(i)}, k)$. Our theoretical analysis reveals that the normal vector evaluated at a wraparound point x in the input space completely encapsulates the projection information of the neuron’s signature. By leveraging the Jacobian matrix, we can formulate a rigorous algebraic mapping between the normal vector and the targeted neuron’s signature (as proven in Theorem 3).

Theorem 3. The Jacobian matrix of the i -th layer can be factorized as $J^{(i)}(x) = \prod_{k=i}^1 (\Lambda^{(k)}(x) W^{(k)})$, where the diagonal activation derivative matrix is given by:

$$\Lambda^{(k)}(x) = \text{diag} \left(\sigma'(z_1^{(k)}(x)), \sigma'(z_2^{(k)}(x)), \dots, \sigma'(z_{d_k}^{(k)}(x)) \right)$$

Furthermore, if x is a wraparound point of neuron $\eta_j^{(i)}$, its corresponding normal vector satisfies

$$n(x) = J^{(i-1)}(x)^T (W_j^{(i)})^T \quad (1)$$

For piecewise linear activation functions (like ReLU), since $\Lambda^{(k)}$ is a diagonal matrix containing only elements 0 or 1, $n(x)$ remains a constant vector within local linear regions. As for smooth activation functions, the derivative $\sigma'(z)$ varies continuously, causing $\Lambda^{(k)}(x)$ to shift continuously with the input x . Its normal vector $n(x)$ will deflect with the spatial position.

First Layer Extraction For the first layer, because the input has not been transformed by any hidden layers, the normal vector n at a wraparound boundary is parallel to the neuron’s weight vector $W_j^{(1)}$. Consequently, the signatures of first-layer neurons can be recovered directly.

To separate first-layer wraparound points from deep-layer ones, we analyze their associated normal vectors. Homologous wraparound points triggered by the same first-layer neuron yield parallel normal vectors. Conversely, because deep-layer wraparound boundaries become highly distorted when mapped back to the input space, their normal vectors exhibit strong irregularity. By clustering these vectors, we exploit this geometric contrast to identify d_1 dense clusters, leaving deep-layer points as discrete outliers. The centroids of these d_1 clusters correspond exactly to the signatures of the first-layer neurons.

Deep Layer Extraction In the signature recovery process for the i -th layer, if the Jacobian matrix $J^{(i-1)}(x)$ achieves full column rank (i.e., $\text{rank}(J^{(i-1)}(x)) = d_{i-1}$), the target neuron’s signature can be directly resolved using equation 1. However, due to the lossy nature of ReLU on negative inputs and the inherent dimension expansion of expansive networks, the Jacobian typically becomes rank-deficient, i.e., $\text{rank}(J^{(i-1)}(x)) < d_{i-1}$. Consequently, the normal vector evaluated at a single wraparound point is frequently insufficient to uniquely determine the neuron’s signature.

Remarkably, non-polynomial activation functions present a non-trivial exception. Because $J(x)$ continuously fluctuates across different points even near a single wraparound boundary, this property guarantees that for non-polynomial activations, a single wraparound point is sufficient to fully recover the neuron’s signature.

Non-Polynomial Activation Functions Extraction Suppose we have identified a wraparound point x_0 associated with the target neuron $\eta_j^{(i)}$. We search its local neighborhood to collect m additional wraparound points $\{x_k\}_{k=1}^m$ lying on the identical wraparound boundary. Because these representations strictly lie on the same signature plane $S(\eta_j^{(i)}, k)$, they satisfy the hyperplane equation $W_j^{(i)} h^{(i-1)}(x_k) + b_j^{(i)} = (2k+1)T_{\max}$.

By calculating the difference vectors $\Delta h_k = h^{(i-1)}(x_k) - h^{(i-1)}(x_0)$ and stacking them, we construct the difference matrix $D = [\Delta h_1 \Delta h_2 \dots \Delta h_m]^T \in \mathbb{R}^{m \times d_{i-1}}$. It satisfies the orthogonality condition $D(W_j^{(i)})^T = \mathbf{0}$. Crucially, Theorem 4 guarantees that for non-polynomial analytic activation functions, the matrix strictly achieves the rank of $\text{rank}(D) = d_{i-1} - 1$. Thus, the exact signature of neuron $\eta_j^{(i)}$ can be directly extracted by solving the one-dimensional null space of D .

Theorem 4. *Let σ be a non-polynomial analytic activation function, and let $x \in B(\eta_j^{(i)}, k)$ be a known wraparound point. Provided that the network’s forward mapping F_{i-1} is non-degenerate within the local R -neighborhood of x , the affine span of the mapped local boundary strictly encompasses the entire signature hyperplane. That is, $\text{Aff}(B_{x,R}^{i-1}) = S(\eta_j^{(i)}, k)$.*

Remark. In actual attack scenarios, we often can only recover a partial subspace V of the neuron signature (i.e., $W_j^{(i)} \in V$). This is mainly because the second-order derivative of unbounded activation functions is approximately zero ($\sigma''(x) \approx 0$) over large intervals. This lack of curvature implies that the Jacobian matrices at adjacent local points are nearly identical, leading to excessively low numerical precision when constructing and solving the difference matrix.

5.2 Solution Space Aggregation Method

To address the issue that a single wraparound point cannot fully recover the neuron signature due to various factors (ReLU, expansive networks, insufficient precision of non-polynomial activation functions), we employ a general solution space aggregation method. This method leverages the partial solution spaces obtained from multiple homologous wraparound points to recover the neuron’s signature.

Consider two distinct wraparound points, x_a and x_b . For notational simplicity, let $J_a = J^{(i-1)}(x_a)$ and $J_b = J^{(i-1)}(x_b)$. We can formally define the visible subspaces V_a and V_b at x_a and x_b as follows:

$$\begin{aligned} V_a &= \text{Row}(J_a^T) \subseteq \mathbb{R}^{d_{i-1}}, \quad \dim(V_a) = r_a \\ V_b &= \text{Row}(J_b^T) \subseteq \mathbb{R}^{d_{i-1}}, \quad \dim(V_b) = r_b \end{aligned}$$

Based on the algebraic relations $J_a^T(W_a)^T = n_a$ and $J_b^T(W_b)^T = n_b$, we can obtain the orthogonal projection of the weight $W_a \in \mathbb{R}^{d_{i-1}}$ (respectively, W_b) onto the visible subspace V_a (respectively, V_b) as $\hat{w}_a = (J_a^T)^\dagger n_a$ (respectively, $\hat{w}_b = (J_b^T)^\dagger n_b$)⁴.

⁴ $(\cdot)^\dagger$ represents the pseudoinverse of the matrix.

Step 1: Verify homologous wraparound points in the target layer. Let $k_{a,b} = \dim(V_a \cap V_b)$ denote the dimension of the intersection between these two visible subspaces. By the fundamental dimension theorem for vector spaces, we have:

$$\dim(V_a \cap V_b) = \dim(V_a) + \dim(V_b) - \dim(V_a + V_b)$$

Because the dimension of the sum space $V_a + V_b$ corresponds to the rank of the horizontally concatenated block matrix $[J_a, J_b]$, we can translate the equation above into a computationally efficient form:

$$k_{a,b} = r_a + r_b - \text{rank}([J_a, J_b])$$

It is insufficient to rely solely on the intersection space reaching a certain dimension for a rigorous verification. We also need to check that W_a and W_b project strictly collinearly onto the shared subspace $V_a \cap V_b$. By Theorem 5, this constraint simply requires us to determine whether the orthogonal projections of $\hat{w}_a$ and $\hat{w}_b$ onto $V_a \cap V_b$ are collinear, a calculation which employs the *Principal Angle Method*. The specific procedure is detailed in Algorithm 1.

Algorithm 1 Homologous Wraparound Point Verification Algorithm

Input: Jacobian matrices J_a, J_b , normal vectors n_a, n_b , thresholds $t, \epsilon_{\text{svd}}, \tau_{\text{score}}$
Output: **true** if two wraparound points are homologous, otherwise **false**

- 1: $\hat{w}_a \leftarrow (J_a^T)^\dagger n_a, \hat{w}_b \leftarrow (J_b^T)^\dagger n_b$ % Orthogonal projections
- 2: $[U_a, \Sigma_a, V_a^T] \leftarrow \text{SVD}(J_a)$
- 3: $[U_b, \Sigma_b, V_b^T] \leftarrow \text{SVD}(J_b)$
- 4: $Q_a \leftarrow (U_a)_{:, \{\sigma_i > \epsilon_{\text{svd}}\}}$
- 5: $Q_b \leftarrow (U_b)_{:, \{\sigma_i > \epsilon_{\text{svd}}\}}$
- 6: $[U, \Sigma, V^T] \leftarrow \text{SVD}(Q_a^T Q_b)$ % Principal Angle Method
- 7: $K \leftarrow \{k \mid \sigma_k > 1 - \epsilon_{\text{svd}}\}$
- 8: **if** $|K| < t$ **then**
- 9: **return false** % Intersection subspace is too small to determine
- 10: $U_{\text{int}} \leftarrow Q_a U_{:,K}$
- 11: $v_a \leftarrow U_{\text{int}}^T \hat{w}_a, v_b \leftarrow U_{\text{int}}^T \hat{w}_b$ % Project onto the intersection space
- 12: $\text{Score} \leftarrow \frac{|v_a \cdot v_b|}{\|v_a\| \|v_b\|}$
- 13: **if** $\text{Score} > \tau_{\text{score}}$: **return true**
- 14: **else** : **return false**

Theorem 5. Let $V_{\text{int}} = V_a \cap V_b$. Assume $\dim(V_{\text{int}}) \geq t$ and no two distinct neuron signature project collinearly onto any t -dimensional subspace. Then, for weights W_a and W_b :

$$P_{\text{int}}(\hat{w}_a) \parallel P_{\text{int}}(\hat{w}_b) \iff W_a = W_b$$

where P_{int} is the orthogonal projection onto V_{int} , and $\parallel$ denotes vector collinearity.

As shown in Table 4, our homologous wraparound point verification algorithm achieves high accuracy across the network. The mechanism effectively prevents false positives, resulting in a 0% incorrect judgment rate across all tested layers and categories. Although a minor 0.006% safe rejection rate occurs in Layer 4 due to the limited intersection space, it does not impact the overall recovery process.

Table 4: Homologous wraparound point verification results evaluated on our CIFAR-10 network architecture with an intersection dimension threshold $t = 8$ and a collinearity constraint limit of 0.99.

Layer	Target Layer Homologous			Deeper Layer Homologous or Non-Homologous
	Correct Identification	Safe Rejection	Incorrect Judgment	Incorrect Judgment
1	100%	0%	0%	0%
2	100%	0%	0%	0%
3	100%	0%	0%	0%
4	99.994%	0.006%	0%	0%
5	100%	0%	0%	0%

Step 2: Aggregate neuron signature. Upon confirming that two wraparound points are homologous, we proceed to aggregate their corresponding signature projections. We first align $\hat{w}_a$ and $\hat{w}_b$ by matching their components within the intersection subspace $V_a \cap V_b$, which determines their relative scaling factor. Following this alignment, we can seamlessly reconstruct the unified projection of the target weight W onto the expanded sum space $V_a + V_b$. The specific procedure is detailed in Algorithm 2.

5.3 Bias and Norm Recovery

Having obtained the signature $\hat{W}_j^{(i)} = c_j^{(i)} W_j^{(i)}$ of neuron $\eta_j^{(i)}$, we then separate its scaling factor and bias. For any wraparound point x , $W_j^{(i)} h^{(i-1)}(x) + b_j^{(i)} = (2k + 1)T_{\max}$ holds for some integer k . If we multiply both sides by $c_j^{(i)}$, we see that the values $\hat{W}_j^{(i)} h^{(i-1)}(x)$ for different k are equally spaced, with the step size equal to $2|c_j^{(i)}|T_{\max}$. By computing this step size, we directly get $|c_j^{(i)}|$. Then we can easily compute the bias $b_j^{(i)}$ and shift it back to the range $(-T_{\max}, T_{\max})$.

Remark. The signature and norm recovery rely solely on the geometric properties of wraparound points in the input space, remaining completely independent

Algorithm 2 Neuron Signature Aggregation Algorithm

Input: Signature projection $\hat{w}_a, \hat{w}_b$, visible subspaces Q_a, Q_b , intersection basis U_{int} , threshold ϵ_{svd}

Output: Aggregated neuron signature w_{agg} in the sum space

- 1: $v_a \leftarrow U_{\text{int}}^T \hat{w}_a, v_b \leftarrow U_{\text{int}}^T \hat{w}_b$
- 2: $s \leftarrow \frac{v_a^T v_b}{\|v_b\|_2^2}$ % Scaling factor
- 3: $Q_{\text{concat}} \leftarrow [Q_a \ Q_b]$
- 4: $[U, \Sigma, V^T] \leftarrow \text{SVD}(Q_{\text{concat}})$
- 5: $Q_{\text{union}} \leftarrow U_{:, \{\sigma_i > \epsilon_{\text{svd}}\}}$
- 6: $M_{\text{proj}} \leftarrow \begin{bmatrix} Q_a^T Q_{\text{union}} \\ Q_b^T Q_{\text{union}} \end{bmatrix}$
- 7: $b_{\text{proj}} \leftarrow \begin{bmatrix} Q_a^T \hat{w}_a \\ s \cdot Q_b^T \hat{w}_b \end{bmatrix}$
- 8: $y_{\text{proj}} \leftarrow M_{\text{proj}}^\dagger b_{\text{proj}}$ % Least squares solution in the union basis
- 9: $w_{\text{agg}} \leftarrow Q_{\text{union}} y_{\text{proj}}$
- 10: **return** w_{agg}

of the model’s specific output values. Crucially, since our method successfully localizes these wraparound points even within the restricted top-1 score scenario, the signature and norm recovery can proceed unhindered.

6 Sign Recovery

In this section, we introduce a statistical method for recovering neuron signs leveraging wraparound points. We first establish the feasibility of our approach when the adversary has access to the network’s raw outputs and compare with the neuron wiggle method. For simplicity, our initial analysis assumes the model’s final output dimension is 1. We then demonstrate the strong scalability of our attack in a more restrictive scenario where the adversary can observe only the top-1 prediction score. Due to space constraints, the improved sign recovery algorithm is deferred to Appendix A.

6.1 Raw Outputs Sign Recovery

The core observation of this method is that the derivatives of unbounded activation functions diverge on the two sides of the wraparound boundary: in the neighborhood interval $((2k+1)T_{\text{max}} - \delta, (2k+1)T_{\text{max}})$, the derivative of the activation function approaches 1; whereas in the interval $((2k+1)T_{\text{max}}, (2k+1)T_{\text{max}} + \delta)$, its derivative drops sharply and approaches 0 ($\alpha \ll 1$ for LeakyReLU).

Consider a wraparound point x^* of the neuron $\eta_j^{(i)}$, the directional derivative of the network output f along an arbitrary unit vector v can be expanded via the chain rule as:

$$\frac{\partial f}{\partial v} = \frac{\partial G_{i+1}}{\partial h^{(i)}} \frac{\partial h^{(i)}}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial h^{(i-1)}} \frac{\partial h^{(i-1)}}{\partial v}$$

Assume the direction vector H aligns with the neuron's activation direction, i.e., $z_j^{(i)}(x + \epsilon H) < z_j^{(i)}(x)$. Then, the directional derivatives on both sides of the wraparound boundary can be expressed as:

$$\left. \frac{\partial f}{\partial v} \right|_{x^* + \epsilon H} = \frac{\partial G_{i+1}(x^* + \epsilon H)}{\partial h^{(i)}} \Lambda^{(i)}(x^* + \epsilon H) W^{(i)} J^{(i-1)}(x^* + \epsilon H) v$$

$$\left. \frac{\partial f}{\partial v} \right|_{x^* - \epsilon H} = \frac{\partial G_{i+1}(x^* - \epsilon H)}{\partial h^{(i)}} \Lambda^{(i)}(x^* - \epsilon H) W^{(i)} J^{(i-1)}(x^* - \epsilon H) v$$

Here, the diagonal activation derivative matrix is:

$$\Lambda^{(i)}(x^* + \epsilon H) = \begin{bmatrix} \sigma'(z_1^{(i)}(x^* + \epsilon H)) & 0 & \cdots & 0 \\ 0 & \sigma'(z_2^{(i)}(x^* + \epsilon H)) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma'(z_{d_i}^{(i)}(x^* + \epsilon H)) \end{bmatrix}$$

$$\Lambda^{(i)}(x^* - \epsilon H) = \begin{bmatrix} \sigma'(z_1^{(i)}(x^* - \epsilon H)) & 0 & \cdots & 0 \\ 0 & \sigma'(z_2^{(i)}(x^* - \epsilon H)) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma'(z_{d_i}^{(i)}(x^* - \epsilon H)) \end{bmatrix}$$

Since x^* is merely the wraparound point of the target neuron in the i -th layer, the mapping of the first $i - 1$ layers of the network remains smoothly continuous in the local neighborhood of x^* . Therefore, taking the limit as $\epsilon \rightarrow 0$, we obtain:

$$\lim_{\epsilon \rightarrow 0} \left. \frac{\partial f}{\partial v} \right|_{x^* + \epsilon H} = y_v^+ = [g_1^+ \ g_2^+ \ \cdots \ g_{d_i}^+] \begin{bmatrix} \tau_1^+ & 0 & \cdots & 0 \\ 0 & \tau_2^+ & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \tau_{d_i}^+ \end{bmatrix} W^{(i)} J^{(i-1)}(x^*) v$$

$$\lim_{\epsilon \rightarrow 0} \left. \frac{\partial f}{\partial v} \right|_{x^* - \epsilon H} = y_v^- = [g_1^- \ g_2^- \ \cdots \ g_{d_i}^-] \begin{bmatrix} \tau_1^- & 0 & \cdots & 0 \\ 0 & \tau_2^- & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \tau_{d_i}^- \end{bmatrix} W^{(i)} J^{(i-1)}(x^*) v$$

In the above equations, except for the target neuron j , the activation states of the other neurons in the same layer remain unchanged in the small neighborhood, i.e., $\tau_k^+ = \tau_k^-$ ($k \neq j$), denoted as τ_k . For the target neuron, its activation state when crossing the wraparound boundary satisfies $\tau_j^+ \approx 1, \tau_j^- \approx 0$ (except for LeakyReLU). To simplify the mathematical derivation, let the forward projection

matrix be $A = W^{(i)}J^{(i-1)}(x^*)$, where A_k denotes the k -th row vector of matrix A . The directional derivatives on both sides can be compactly expressed as:

$$y_v^+ = \sum_{k=1}^{d_i} g_k^+ \tau_k^+ A_k v, \quad y_v^- = \sum_{k=1}^{d_i} g_k^- \tau_k^- A_k v$$

Due to the significant magnitude difference between τ_j^+ and τ_j^- , the sign recovery problem of the neuron can be equivalently transformed into an optimization problem of extracting the target signal from complex interference signals: where the target neuron signal is $A_j v$, and the background interference signals caused by other neurons in the same layer are $A_k v$ ($k \neq j$). Our core task is to find a specific probe direction v that maximizes the difference between the target signal $A_j v$ and the interference signals. This ensures $|y_v^+| \gg |y_v^-|$, so we can determine the neuron's sign by comparing the absolute directional derivatives on both sides.

To increase the signal difference, the neuron wiggle method [4] is to find the vector that maximizes the target signal $A_j v = W_j^{(i)} J^{(i-1)}(x^*) v$, i.e., selecting $v_1 = J^{(i-1)}(x^*)^\dagger (W_j^{(i)})^T$. In contrast, our paper proposes constructing the probe direction $v_2 = A^\dagger e_j$.

Under a contractive non-ReLU network architecture, the algebraic identities $J(x)J(x)^\dagger = I$ and $AA^\dagger = I$ hold. In the neuron wiggle method, using the direction v_1 , the interference signals equal the inner product of the network weights, i.e., $A_k v_1 = \langle W_k^{(i)}, W_j^{(i)} \rangle$. The directional derivative is $|y_{v_1}^+| = |g_j^+ \tau_j^+ \|W_j^{(i)}\|^2 + \sum_{k \neq j} g_k^+ \tau_k^+ \langle W_k^{(i)}, W_j^{(i)} \rangle|$. In complex networks, even when neurons k and j are nearly orthogonal ($\langle W_k^{(i)}, W_j^{(i)} \rangle \approx 0$), deep transfer gradients satisfying $|c_k| \gg |c_j|$ can cause the massive cumulative interference $|\sum_{k \neq j} g_k^+ \tau_k^+ \langle W_k^{(i)}, W_j^{(i)} \rangle|$ to submerge the weak target signal, leading to the failure of sign recovery.

Our approach utilizes the crafted direction $v_2 = A^\dagger e_j$. This guarantees that the target neuron responds with $A_j v_2 = 1$, while all other neurons $k \neq j$ give zero response $A_k v_2 = 0$. Thus, the directional derivative is perfectly purified into $y_{v_2}^+ = g_j^+ \tau_j^+$. A correct sign guess produces $|y_{v_2}^+| \gg 0$, whereas an incorrect guess results in $y_{v_2}^- = 0$, enabling direct sign recovery. In what follows, a rigorous probabilistic proof shows that our method comprehensively outperforms the neuron wiggle approach, regardless of the network architecture or activation function.

Definition 13. *The success probability of the sign recovery attack for a probe direction v is defined as $p = \Pr[|y_v^+| > |y_v^-|]$. Furthermore, the Signal-to-Interference Ratio (SIR) for a probe direction v is formulated as:*

$$\text{SIR}(v) = \frac{(A_j v)^2}{\sum_{k \neq j} (A_k v)^2}$$

In order to analyze extremely complex networks, it is naturally assumed that the deep coefficients g_k^+ and g_k^- are independent and identically distributed

random variables following a zero-mean normal distribution. Furthermore, we consider the worst-case scenario where the activation states across the boundary satisfy $\tau_j^+ = 1$, $\tau_j^- = \alpha$, and $\tau_k = 1$ for all $k \neq j$. Then, an exact analytical solution for the sign recovery success probability can be rigorously derived, as formalized in Theorem 6.

Theorem 6. *Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = \alpha$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients satisfy $g_k^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2), g_k^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. Under these conditions, the success probability is*

$$p = \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}} \right)$$

Observing the above analytical solution, for unbounded activation functions (where the inactive side derivative ratio is $\alpha < 1$), the success probability p is a strictly monotonically increasing function of $\text{SIR}(v)$. Specifically, as $\text{SIR}(v) \rightarrow \infty$, the success rate $p \rightarrow 1$.

Furthermore, even if the deep gradient coefficients no longer follow an ideal normal distribution, Theorem 7 guarantees that, in the asymptotic sense of a wide network with a sufficiently high neuron dimension, the success probability of sign recovery still maintains a monotonically increasing relationship with $\text{SIR}(v)$.

Theorem 7. *Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = 0$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients g_k^+ and g_k^- ($k \neq j$) are independent and identically distributed with finite variance σ_+^2 and σ_-^2 . Under the condition of asymptotic stability $\lim_{d_i \rightarrow \infty} \frac{\max_{k \neq j} (A_k v)^2}{\sum_{k \neq j} (A_k v)^2} = 0$, the asymptotic success probability is:*

$$\lim_{d_i \rightarrow \infty} p = \Phi_2(\Gamma, \Gamma; \rho) + \Phi_2(-\Gamma, -\Gamma; \rho)$$

where Φ_2 denotes the standard bivariate normal cumulative distribution function, the integral boundary is $\Gamma = g_j^+ \sqrt{\frac{\text{SIR}(v)}{\sigma_+^2 + \sigma_-^2}}$, and the correlation coefficient is $\rho = \frac{\sigma_+^2 - \sigma_-^2}{\sigma_+^2 + \sigma_-^2}$. Furthermore, this asymptotic probability strictly monotonically increases with $\text{SIR}(v)$.

In summary, $\text{SIR}(v)$ quantifies the ability of the target neuron signal to break through the background interference signals. Therefore, our goal is to choose the direction v that maximizes $\text{SIR}(v)$. Theorem 8 shows that the probe direction $v = A^\dagger e_j$ in our method achieves this global maximum.

Theorem 8. *The Signal-to-Interference Ratio $\text{SIR}(v) = \frac{(A_j v)^2}{\sum_{k \neq j} (A_k v)^2}$ is maximized in the direction $v = A^\dagger e_j$.*

Based on the above analysis, the active side is more likely to yield a larger directional derivative magnitude than the inactive side. Therefore, we adopt a statistical voting approach similar to those in [4,5] to determine the sign of the neuron. Algorithm 3 formalizes our dynamic sign recovery procedure: for each evaluated wraparound point, the algorithm computes and compares the absolute magnitudes of the directional derivatives on both sides of the boundary, and stops early once the *confidence level* (introduced in subsection 7.3) for a particular sign reaches a target threshold.

Algorithm 3 Basic Sign Recovery Algorithm

Input: Recovered weights matrix $\hat{W}^{(i)}$, oracle $\mathcal{O}$, parameters ϵ, δ , threshold $\text{CL}_{\text{target}}$

Output: **true** if the sign of the target neuron is right, otherwise **false**

```

1: votes+  $\leftarrow$  0, votes-  $\leftarrow$  0, CL  $\leftarrow$  0
2: while CL < CLtarget do
3:    $x \leftarrow \text{FindWraparoundPoint}(i, j)$ 
4:    $N \leftarrow N + 1$ 
5:    $A \leftarrow W^{(i)} J^{(i-1)}(x)$ 
6:    $v \leftarrow A^\dagger e_j$ 
7:    $x_{\text{pos}} \leftarrow x - \epsilon v$ ,  $x_{\text{neg}} \leftarrow x + \epsilon v$ 
8:    $y_v^+ \leftarrow (\mathcal{O}(x_{\text{pos}}) - \mathcal{O}(x_{\text{pos}} - \delta v)) / \delta$ 
9:    $y_v^- \leftarrow (\mathcal{O}(x_{\text{neg}} + \delta v) - \mathcal{O}(x_{\text{neg}})) / \delta$ 
10:  if  $|y_v^+| > |y_v^-|$  : votes+  $\leftarrow$  votes+ + 1
11:  else : votes-  $\leftarrow$  votes- + 1
12:  CL  $\leftarrow \text{ComputeConfidenceLevel}(\text{votes}_+, \text{votes}_-)$ 
13: if votes+ > votes- : return true
14: else : return false

```

6.2 Top-1 Score Sign Recovery

The sign recovery phase depends heavily on the directional derivative of the output function along a probe vector v . When the oracle provides only the top-1 label and its probability, we can only observe $\frac{\partial p_{\text{top}}}{\partial v}$ instead of $\frac{\partial f}{\partial v}$ (the directional derivative of raw outputs). Therefore, a detailed theoretical analysis of $\frac{\partial p_{\text{top}}}{\partial v}$ on both sides of the wraparound boundary is necessary.

Let G^+ and G^- denote the asymptotic deep coefficients corresponding to the raw outputs. We define the limiting probability vectors on the active and inactive sides of the wraparound boundary as $p^+ = [p_1^+, p_2^+, \dots, p_{d_{r+1}}^+]^T$ and $p^- = [p_1^-, p_2^-, \dots, p_{d_{r+1}}^-]^T$, respectively. The limiting directional derivatives of the top-1 probability on the active and inactive sides are denoted as

$$p_v^+ = \lim_{\epsilon \rightarrow 0} \frac{\partial p_{\text{top}^+}}{\partial v} \Big|_{x^* + \epsilon H} \quad p_v^- = \lim_{\epsilon \rightarrow 0} \frac{\partial p_{\text{top}^-}}{\partial v} \Big|_{x^* - \epsilon H}$$

Building upon Theorem 9, we demonstrate that our sign recovery methodology can be seamlessly extended to the restrictive top-1 score scenario by evaluating and comparing the directional derivatives of these probabilities.

Definition 14. The success probability of the sign recovery attack in the top-1 score scenario for a probe direction v is defined as $p' = \Pr[|p_v^+| > |p_v^-|]$.

Theorem 9. Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = \alpha$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients satisfy $G_{l,k}^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2), G_{l,k}^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. Under these conditions, the success probability in the top-1 score scenario is

$$p' = \frac{2}{\pi} \arctan \left(\frac{\sigma_+ p_{top^+}^+}{\sigma_- p_{top^-}^-} \sqrt{\frac{\sum_{l \neq top^+} (p_l^+)^2}{\sum_{l \neq top^-} (p_l^-)^2} \cdot \frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}} \right)$$

Remark. The output probability vectors p^+ and p^- on the two sides of the boundary are typically unequal, which may affect the stability of sign recovery. However, the improved method presented in Appendix A can systematically resolve this issue.

7 Experiments

7.1 Experiment Setup

The experimental setup is meticulously designed to simulate an operational MPC environment over a finite ring with modulus $M = 2^{64}$ and a fixed-point scaling factor of $S = 26$ [29,30]. To accelerate the evaluation of the MPC protocols, all simulated experiments are executed in plaintext. Furthermore, the layer-wise attack is evaluated under the assumption that all preceding layers have already been perfectly extracted ⁵.

Our CIFAR10 Network. The target CIFAR10 network models are fully connected neural networks adopting a structural configuration of 3072-256x4-10. Specifically, the architecture comprises 3,072 input neurons, four identically sized hidden layers containing 256 neurons each, and terminates in a 10-neuron output layer. To comprehensively evaluate the generalizability of the attack, the hidden neurons employ the eight different unbounded activation functions introduced previously (i.e., ReLU, LeakyReLU, ELU, SELU, Softplus, Swish, GELU, and Mish). The models are trained following a methodology consistent with established baselines [4,5]. Under this unified setup, the target networks achieve test accuracies ranging from 53.22% to 55.37% on the CIFAR10 dataset, depending on the specific activation function employed.

⁵ Our full code is available at https://anonymous.4open.science/r/PPML_Model_Extraction_Attack

Table 5: Wraparound points required for signature recovery across different activation functions on our CIFAR10 and random networks.

Layer	ReLU		LeakyReLU ($\alpha = 0.1$)		Non-Linear	
	CIFAR10	Random	CIFAR10	Random	CIFAR10	Random
1	1.00	1.00	1.00	1.00	1.00 ± 0.00	1.00 ± 0.00
2	9.48	10.17	1.00	3.25	9.45 ± 0.12	10.04 ± 0.15
3	19.43	28.06	1.00	-	20.52 ± 3.01	26.16 ± 3.38
4	9.57	14.76	2.00	-	11.25 ± 1.70	14.36 ± 1.07
5	9.80	11.00	2.00	-	11.98 ± 1.47	9.40 ± 0.40

7.2 Signature Recovery

After obtaining a sufficient number of wraparound points, we need to compute the normal vector of the wraparound boundary at each point and then perform clustering based on whether they are homologous. We employ a *dynamic weight aggregation* method: each time a normal vector is extracted, it is compared against a dynamically maintained set of cluster centers. If the newly extracted vector is homologous to an existing cluster center, it is aggregated to update and refine that center. Conversely, if no existing center is homologous to the point, the algorithm automatically instantiates a new cluster center.

Activation Functions. The eight activation functions evaluated fall into three categories: ReLU, LeakyReLU, and non-linear activations. As shown in Table 5, the number of wraparound points required for signature recovery depends on both the layer depth and the activation type. For the first layer, a single wraparound point is sufficient across all activation functions. In shallow layers, LeakyReLU still requires only one point, because its Jacobian matrix maintains full rank in our CIFAR10 contractive network. However, for deeper layers (layer 4 and 5), the ill-conditioned Jacobian matrices cause precision issues, requiring two wraparound points to recover the signatures. In contrast, ReLU and non-linear activations require an average of 9 to 20 wraparound points for successful deep-layer recovery.

Expansive Architectures. To evaluate our algorithm on expansive architectures, we test a randomly generated 100-4x200-10 network. As shown in Table 5, we successfully recover the signatures for all layers in networks employing ReLU and other non-linear activation functions, where complete recovery of deep-layer parameters typically requires 10 to 28 wraparound points. However, for networks utilizing LeakyReLU, this method is only effective in recovering parameters for the first two layers. This limitation fundamentally stems from LeakyReLU’s property of having a non-zero gradient across its entire domain. In layers 3 to 5, following dense matrix multiplications, homologous wraparound points sampled at different locations in the input space activate completely distinct background

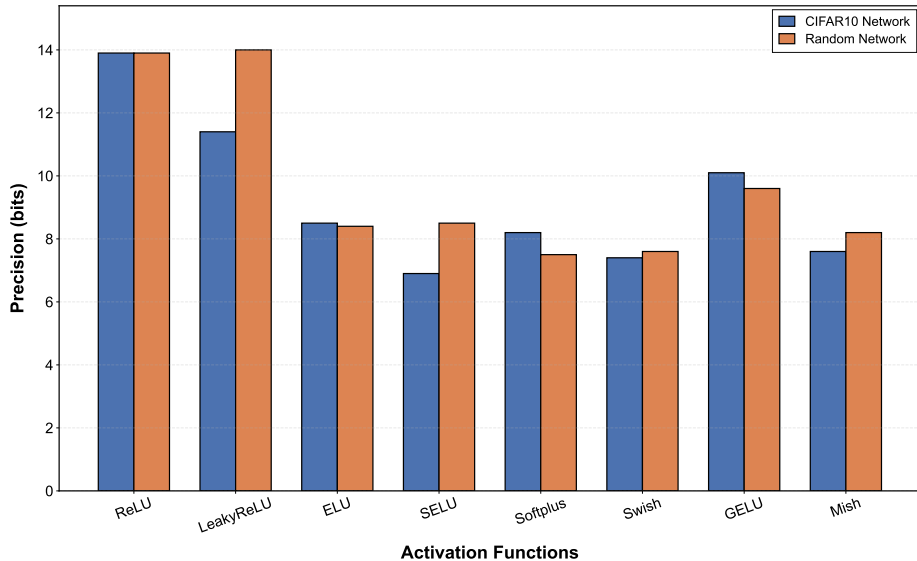


Fig. 4: Bits of precision of the extracted weights on our CIFAR10 and random network (first two layers for LeakyReLU-based random network).

redundant neurons. This highly position-dependent global noise severely corrupts their respective visible subspaces. When attempting to compute the intersection of these subspaces, their principal angles increase sharply due to severe deflections within a high-dimension space. Ultimately, the intersection computation rapidly degenerates into a zero-dimensional null space, making further subspace aggregation and accurate weight recovery ineffective. Recovering the expansive network based on LeakyReLU remains a future research direction.

Extraction Fidelity. As illustrated in Figure 4, the choice of activation function impacts extraction precision, a trend that remains consistent across both networks. Specifically, piecewise linear activation functions exhibit extraction vulnerability: ReLU achieves an exceptionally high precision of approximately 14 bits in both settings, with LeakyReLU maintaining a similarly high level. In contrast, non-linear activation functions possess inherent non-zero curvature, which introduces unavoidable systematic truncation errors during the computation of local Jacobian matrices and normal vectors. Consequently, this mathematical property compresses the extraction precision to a tighter range of 7 to 10 bits.

7.3 Sign Recovery

After recovering the signature and the magnitude, we can obtain the set of wraparound points for any neuron in this layer. We then apply our sign recovery

Table 6: Wraparound points required for sign recovery across different activation functions on our CIFAR10 and random networks.

Layer	ReLU		LeakyReLU ($\alpha = 0.1$)		Non-Linear	
	CIFAR10	Random	CIFAR10	Random	CIFAR10	Random
1	5.00	6.97	5.00	6.93	5.00 ± 0.00	6.85 ± 0.36
2	11.92	6.54	5.00	5.83	5.27 ± 0.10	6.43 ± 0.17
3	9.51	6.13	5.02	5.53	5.65 ± 0.16	5.89 ± 0.12
4	5.42	6.53	9.51	5.43	5.12 ± 0.09	6.13 ± 0.32

ery algorithm. Once the confidence level exceeds a specified threshold, we can determine the sign of the neuron and proceed to recover the sign of the next neuron.

Confidence Level. To statistically validate the sign recovery results, we evaluate the confidence level (CL) using a binomial distribution model. Suppose we evaluate N wraparound points for a given neuron, where the majority sign receives $M = \max(\text{votes}_+, \text{votes}_-)$ votes. Under the assumption of a random guess, the confidence level is defined as $\text{CL} = 1 - \Pr[X \geq M]$, where $X \sim \text{Binomial}(N, 0.5)$. To compute this efficiently, we apply a hybrid approach based on the sample size N . For small samples ($N < 10$), we use a precomputed lookup table of exact binomial probabilities to obtain the precise CL. For larger samples ($N \geq 10$), we use Hoeffding’s inequality [13] to provide a computationally efficient and conservative lower bound: $\text{CL} \geq 1 - \exp\left(-\frac{2(M-N/2)^2}{N}\right)$. We make the sign decision if the CL exceeds a preset threshold (e.g., 95%). If all votes are consistent, only $N = 5$ points are required to achieve a 95% CL. However, if the voting probability is close to random (e.g., $M/N = 0.55$), achieving the same 95% threshold requires a significantly larger sample size of at least $N = 600$ points.

Activation Functions. Our sign recovery algorithm achieves a 100% success rate across all recoverable layers in the eight evaluated networks, encompassing both CIFAR-10 and random models. As demonstrated in Table 6, recovering the parameter signs for CIFAR-10 networks utilizing LeakyReLU generally requires a mere 5 wraparound points. The sole exception is the fourth layer of the LeakyReLU network, where an ill-conditioned Jacobian matrix necessitates a larger number of points. Conversely, for deeper layers employing non-LeakyReLU activation functions, we observe an intriguing trend: the recovery process actually becomes more efficient as network depth increases. A notable difference in expansive networks is that neuron signs cannot be recovered with an average of five points.

Table 7: Comparison of sign recovery results evaluated across CIFAR10 networks with different activation functions. Each network consists of 1024 hidden-layer neurons tested with a maximum of 1000 attempts and a target confidence level of 0.95.

Activation	Method	$\alpha \geq 0.95$		$\alpha < 0.95$		Average Points
		Correct	Incorrect	Correct	Incorrect	
ReLU	Neuron Wiggle	988/1024	26/1024	7/1024	3/1024	24.82
	Ours	1024/1024	0/1024	0/1024	0/1024	6.61 (0.26 $\times$)
LeakyReLU	Neuron Wiggle	1020/1024	4/1024	0/1024	0/1024	7.25
	Ours	1024/1024	0/1024	0/1024	0/1024	5.09 (0.70 $\times$)
Non-Linear	Neuron Wiggle	1016/1024	5/1024	1/1024	2/1024	14.58
	Ours	1024/1024	0/1024	0/1024	0/1024	5.22 (0.36 $\times$)

Comparison with Neuron Wiggle Method. Table 7 presents a detailed comparison between our approach and the neuron wiggle method on CIFAR-10 networks. For the hidden layers, our algorithm correctly recovers the sign of every neuron, achieving a 95% confidence level across all neurons within 1,000 queries. Under the same query budget, the neuron wiggle method fails to reach this threshold for several neurons. More critically, even when the neuron wiggle method exceeds the confidence threshold, it still produces 4 to 29 incorrect signs. These errors are sufficient to derail the subsequent deep-layer parameters recovery. Furthermore, our algorithm requires an average of fewer than 7 wraparound points per neuron, whereas the neuron wiggle method demands over 24 points for ReLU neurons. Overall, our approach effectively reduces query complexity by 30% to 74%.

Top-1 Score Scenario. In the restricted top-1 scenario, accessing only the maximum softmax-normalized probability severely degrades the stability of the sign recovery process. To mitigate this and enhance robustness, we enforce a strict baseline requiring the evaluation of at least 20 wraparound points before making the sign decision. As shown in Figure 8, under this configuration, we correctly recover the signs of all 14,592 neurons, spanning two network architectures and eight activation functions. Although 5 neurons initially fail to reach the targeted 95% confidence threshold within the maximum allowed attempts, subsequent isolated analysis enables them to ultimately meet this confidence criterion.

8 Conclusion and Remarks

This paper addresses two major challenges in the cryptanalytic extraction of neural network parameters: smooth activation functions and expansive network architectures, and proposes a better sign recovery algorithm. Under the privacy-preserving inference model, we successfully extract parameters from all unbounded activation functions and expansive networks in polynomial time, and extend our method to the restricted top-1 score scenario. Empirically, we successfully extract all parameters on CIFAR10 networks with nearly one million parameters across eight different activations. However, our method relies on arithmetic wraparound and therefore cannot extract parameters from networks with bounded activations (e.g., Sigmoid, Tanh), where severe vanishing gradients suppress modular wraparounds. Additionally, neurons with exceptionally small weight norms ($|w| \ll 1$) further evade extraction as they cannot accumulate enough value to trigger a wraparound. Consequently, attacking networks with bounded smooth activations and exploring efficient and robust MPC protocols to defend against wraparound-based extraction attacks remain important future research directions.

References

1. Baum, E.B.: A polynomial time algorithm that learns two hidden unit nets. *Neural Computation* **2**(4), 510–522 (1990)
2. Baum, E.B.: Neural net algorithms that learn in polynomial time from examples and queries. *IEEE Transactions on Neural Networks* **2**(1), 5–19 (1991)
3. Blum, A., Rivest, R.: Training a 3-node neural network is np-complete. *Advances in neural information processing systems* **1** (1988)
4. Canales-Martínez, I.A., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Satpute, N., Shamir, A.: Polynomial time cryptanalytic extraction of neural network models. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 3–33. Springer (2024)
5. Carlini, N., Chávez-Saab, J., Hambitzer, A., Rodríguez-Henríquez, F., Shamir, A.: Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 364–396. Springer (2025)
6. Carlini, N., Jagielski, M., Mironov, I.: Cryptanalytic extraction of neural network models. In: *Annual international cryptology conference*. pp. 189–218. Springer (2020)
7. Chen, Y., Dong, X., Guo, J., Shen, Y., Wang, A., Wang, X.: Hard-label cryptanalytic extraction of neural network models. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 207–236. Springer (2024)
8. Chen, Y., Dong, X., Ma, R., Shen, Y., Wang, A., Yu, H., Wang, X.: Delving into cryptanalytic extraction of prelu neural networks. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 576–607. Springer (2025)
9. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: *Annual cryptology conference*. pp. 643–662. Springer (2012)
10. Daniely, A., Granot, E.: An exact poly-time membership-queries algorithm for extracting a three-layer relu network. In: *The Eleventh International Conference on Learning Representations*
11. Foerster, H., Mullins, R., Shumailov, I., Hayes, J.: Beyond slow signs in high-fidelity model extraction. *Advances in Neural Information Processing Systems* **37**, 19496–19522 (2024)
12. Hancock, T.R., Golea, M., Marchand, M.: Learning nonoverlapping perceptron networks from examples and membership queries. *Machine Learning* **16**(3), 161–183 (1994)
13. Hoeffding, W.: Probability inequalities for sums of bounded random variables. *Journal of the American statistical association* **58**(301), 13–30 (1963)
14. Jagielski, M., Carlini, N., Berthelot, D., Kurakin, A., Papernot, N.: High accuracy and high fidelity extraction of neural networks. In: *29th USENIX security symposium (USENIX Security 20)*. pp. 1345–1362 (2020)
15. Lehmkuhl, R., Mishra, P., Srinivasan, A., Popa, R.A.: Muse: Secure inference resilient to malicious clients. In: *30th USENIX Security Symposium (USENIX Security 21)*. pp. 2201–2218 (2021)
16. Lin, V.Y., Pinkus, A.: Fundamentality of ridge functions. *Journal of Approximation Theory* **75**(3), 295–311 (1993)

17. Liu, J., Juuti, M., Lu, Y., Asokan, N.: Oblivious neural network predictions via minionn transformations. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security. pp. 619–631 (2017)
18. Martinelli, F., Şimşek, B., Gerstner, W., Brea, J.: Expand-and-cluster: parameter recovery of neural networks. In: Proceedings of the 41st International Conference on Machine Learning. pp. 34895–34919 (2024)
19. Milli, S., Schmidt, L., Dragan, A.D., Hardt, M.: Model reconstruction from model explanations. In: Proceedings of the Conference on Fairness, Accountability, and Transparency. pp. 1–9 (2019)
20. Mohassel, P., Rindal, P.: Aby3: A mixed protocol framework for machine learning. In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security. pp. 35–52 (2018)
21. Mohassel, P., Zhang, Y.: Secureml: A system for scalable privacy-preserving machine learning. In: 2017 IEEE symposium on security and privacy (SP). pp. 19–38. IEEE (2017)
22. Rathee, D., Rathee, M., Goli, R.K.K., Gupta, D., Sharma, R., Chandran, N., Rastogi, A.: Sirnn: A math library for secure rnn inference. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1003–1020. IEEE (2021)
23. Rathee, D., Rathee, M., Kumar, N., Chandran, N., Gupta, D., Rastogi, A., Sharma, R.: Cryptflow2: Practical 2-party secure inference. In: Proceedings of the 2020 ACM SIGSAC conference on computer and communications security. pp. 325–342 (2020)
24. Reith, R.N., Schneider, T., Tkachenko, O.: Efficiently stealing your machine learning models. In: Proceedings of the 18th ACM Workshop on Privacy in the Electronic Society. pp. 198–210 (2019)
25. Tan, S., Knott, B., Tian, Y., Wu, D.J.: Cryptgpu: Fast privacy-preserving machine learning on the gpu. In: 2021 IEEE Symposium on Security and Privacy (SP). pp. 1021–1038. IEEE (2021)
26. Tramèr, F., Zhang, F., Juels, A., Reiter, M.K., Ristenpart, T.: Stealing machine learning models via prediction {APIs}. In: 25th USENIX security symposium (USENIX Security 16). pp. 601–618 (2016)
27. Wagh, S., Gupta, D., Chandran, N.: Securenn: 3-party secure computation for neural network training. Proceedings on Privacy Enhancing Technologies (2019)
28. Wagh, S., Tople, S., Benhamouda, F., Kushilevitz, E., Mittal, P., Rabin, T.: Falcon: Honest-majority maliciously secure framework for private deep learning. Proceedings on Privacy Enhancing Technologies **1**, 188–208 (2021)
29. Watson, J.L., Wagh, S., Popa, R.A.: Piranha: A {GPU} platform for secure computation. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 827–844 (2022)
30. Zhou, L., Zhang, B., Wang, Z., Lu, T., Song, Q., Zhang, S., Cui, H., Yu, Y.: On probabilistic truncation in privacy-preserving machine learning. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 39, pp. 22955–22964 (2025)

A Improved Sign Recovery Algorithm

The improved sign recovery algorithm includes two parts: one is to select a noise direction for comparison (A.1), and the other is to leverage additional known information to obtain a superior probing direction (A.2). The specific algorithm is shown in Algorithm 4.

A.1 Improvement via Utilizing Noise Direction

In the previous analysis, directly comparing the derivatives $|y_v^+|$ and $|y_v^-|$ across the wraparound boundary causes a large error. When the input crosses a neuron’s wraparound boundary, its output suddenly jumps by up to $T_{\max}$. This sudden jump creates a domino effect, changing the activation states of many deeper neurons. As a result, the gradients on the two sides of the boundary become very different ($g_i^+ \neq g_i^-$). If the positive variance is much smaller than the negative one ($\sigma_+ \ll \sigma_-$), our success probability p drops significantly. This problem is even worse in the top-1 score scenario. Because the probabilities on the two sides are different ($p^+ \neq p^-$), they multiply the gradients unevenly, creating a similar error. To fix the errors caused by these deep network changes and the uneven top-1 probabilities, we propose a new, more robust method. Instead of comparing the two sides directly, we calculate a ratio using signals from the same side. This strategy requires two specific probe directions:

- **Probe direction v :** We use the optimal direction $v = A^\dagger e_j$. This direction strongly excites the target neuron while minimizing background noise.
- **Noise direction u :** This direction is designed to not excite the target neuron at all (i.e. $A_j u = 0$).

Considering the worst-case scenario where the activation states across the boundary satisfy $\tau_j^+ = 1$, $\tau_j^- = \alpha$, and $\tau_k = 1$ for all $k \neq j$, we measure the derivatives along both directions v and u on the positive side:

$$y_v^+ = g_j^+(A_j v) + \sum_{k \neq j} g_k^+(A_k v)$$

$$y_u^+ = g_j^+(A_j u) + \sum_{k \neq j} g_k^+(A_k u) = \sum_{k \neq j} g_k^+(A_k u)$$

We do the same on the negative side to get y_v^- and y_u^- . Because the noise direction u completely ignores the target neuron, y_u^+ and y_u^- only contain background noise. This gives us a reliable baseline for the noise on each side. We then calculate the ratio of the signal to the noise for each side:

$$\tau^+ = \frac{|y_v^+|}{|y_u^+|}, \quad \tau^- = \frac{|y_v^-|}{|y_u^-|}$$

If our sign guess is correct, the target neuron is active. Because the probe direction v focuses on the target neuron ($A_j v \approx 1$) and reduces noise, the signal

is strong. This makes the ratio τ^+ much larger than 1. On the other hand, if our guess is wrong, the target neuron is inactive. The signal mostly disappears, leaving only random noise on both the top and bottom of the fraction. This makes the ratio $\tau^- \approx 1$ (assuming α is small).

Definition 15. *The success probability of the sign recovery based on the noise direction is defined as $p'' = P(\tau^+ > \tau^-)$.*

Theorem 10. *Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = \alpha$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients satisfy $g_k^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2), g_k^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. Under these conditions, the success probability based on the noise direction is*

$$p'' = 1 - \frac{4}{\pi^2} \int_0^\infty \frac{\arctan(kt)}{t^2 + 1} dt$$

where the simplification constant is $k = \sqrt{\frac{\alpha^2 \text{SIR}(v) + 1}{\text{SIR}(v) + 1}}$.

The result of Theorem 10 shows that the success probability p'' increases with $\text{SIR}(v)$ and eventually reaches 1. Most importantly, the formula proves that p'' is no longer affected by the unbalanced variances σ_+ and σ_- . As a result, this method completely solves the problems in the top-1 score scenario. By taking the ratio of the probe direction v and the noise direction u on the same side, the unbalanced scaling simply cancels out during division. This removes the errors caused by both the deep network jumps ($\sigma_+ \ll \sigma_-$) and the uneven top-1 probabilities ($p^+ \neq p^-$), allowing us to recover parameters accurately even in strict and noisy environments.

A.2 Improvement via Refined Probe Direction

Earlier, we assumed the worst-case scenario where all other background neurons were fully active ($\tau_k = 1$ for $k \neq j$). However, in practice, we have more information about how τ_k behaves.

As we recover neurons one by one in the target layer, we already know the exact signs of some neurons. Let I be the group of neurons whose signs are successfully recovered, and let I' be the group of neurons whose signs are still unknown (excluding the target neuron j). With this information, the probability distribution of τ_k is:

$$\tau_k = \begin{cases} \sigma'(z_k^{(i)}(x^*)), & \text{if } k \in I, \\ \sigma'(z_k^{(i)}(x^*)), & \text{with probability } \frac{1}{2} \text{ if } k \in I', \\ \sigma'(-z_k^{(i)}(x^*)), & \text{with probability } \frac{1}{2} \text{ if } k \in I'. \end{cases}$$

Using this distribution, we can calculate the expected value $w_k = \mathbb{E}[\tau_k^2]$ as:

$$w_k = \mathbb{E}[\tau_k^2] = \begin{cases} (\sigma'(z_k^{(i)}(x^*)))^2, & \text{if } k \in I \\ \frac{1}{2}(\sigma'(z_k^{(i)}(x^*)))^2 + \frac{1}{2}(\sigma'(-z_k^{(i)}(x^*)))^2, & \text{if } k \in I' \end{cases}$$

Theorem 11. Assume the target neuron's activation states across the boundary are $\tau_j^+ = 1$ and $\tau_j^- = \alpha$. Also, assume the deep coefficients satisfy $g_k^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2)$ and $g_k^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. If the network satisfies the following stability condition as it grows larger

$$\lim_{d_i \rightarrow \infty} \frac{\sum_{k \neq j} (A_k v)^4 \text{Var}(\tau_k^2)}{\left(\sum_{k \neq j} (A_k v)^2 \mathbb{E}[\tau_k^2] \right)^2} = 0$$

Then the asymptotic success probability is

$$\lim_{d_i \rightarrow \infty} p = \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{(A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2}{\alpha^2 (A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2}} \right)$$

Furthermore, this success probability reaches its maximum value when we use the direction $v = (A')^\dagger e_j$, where $A'_j = A_j$, and $A'_k = \sqrt{w_k} A_k$ ($\forall k \neq j$).

Theorem 11 gives us a better direction $v = (A')^\dagger e_j$. We can directly plug this new direction into the method we described in Section A.1.

Algorithm 4 Improved Sign Recovery Algorithm

Input: Recovered weights matrix $\hat{W}^{(i)}$, oracle $\mathcal{O}$, recovered neurons I and unrecovered neurons I' , parameters ϵ, δ , threshold $\text{CL}_{\text{target}}$

Output: **true** if the sign of target neuron is right, otherwise **false**

```

1: votes+  $\leftarrow$  0, votes-  $\leftarrow$  0, CL  $\leftarrow$  0
2: while CL < CLtarget do
3:    $x \leftarrow \text{FindWraparoundPoint}(i, j)$ 
4:    $N \leftarrow N + 1$ 
5:    $A \leftarrow \hat{W}^{(i)} J^{(i-1)}(x)$ 
6:   for all  $k \in I \cup I'$  do
7:     if  $k \in I$  :  $w_k \leftarrow (\sigma'(z_k^{(i)}(x)))^2$ 
8:     else :  $w_k \leftarrow ((\sigma'(z_k^{(i)}(x)))^2 + (\sigma'(-z_k^{(i)}(x)))^2)/2$ 
9:      $A'_k \leftarrow \sqrt{w_k} A_k$ 
10:   $A'_j \leftarrow A_j$ 
11:   $v \leftarrow (A')^\dagger e_j$ 
12:   $u \stackrel{\$}{\leftarrow} \{u \mid A_j u = 0\}$ 
13:   $x_{\text{pos}} \leftarrow x - \epsilon v$ ,  $x_{\text{neg}} \leftarrow x + \epsilon v$ 
14:   $y_v^+ \leftarrow (\mathcal{O}(x_{\text{pos}} + \delta v) - \mathcal{O}(x_{\text{pos}}))/\delta$ ,  $y_u^+ \leftarrow (\mathcal{O}(x_{\text{pos}} + \delta u) - \mathcal{O}(x_{\text{pos}}))/\delta$ 
15:   $y_v^- \leftarrow (\mathcal{O}(x_{\text{neg}} + \delta v) - \mathcal{O}(x_{\text{neg}}))/\delta$ ,  $y_u^- \leftarrow (\mathcal{O}(x_{\text{neg}} + \delta u) - \mathcal{O}(x_{\text{neg}}))/\delta$ 
16:   $\tau^+ \leftarrow |y_v^+|/|y_u^+|$ ,  $\tau^- \leftarrow |y_v^-|/|y_u^-|$ 
17:  if  $\tau^+ > \tau^-$  : votes+  $\leftarrow$  votes+ + 1
18:  else : votes-  $\leftarrow$  votes- + 1
19:  CL  $\leftarrow \text{ComputeConfidenceLevel}(\text{votes}_+, \text{votes}_-)$ 
20: if votes+ > votes- : return true
21: else : return false

```

B Experiment Results

Table 8: Sign recovery analysis in top-1 score scenario.

Network		RELU	LeakyReLU	ELU	SELU
CIFAR10	Average Points	55.3	34.4	36.3	38.5
	$\alpha < 0.95$	2/1024	1/1024	0/1024	0/1024
Random	Average Points	55.3	34.4	36.3	38.5
	$\alpha < 0.95$	0/800	0/800	0/800	1/800
Network		Softplus	Swish	GELU	Mish
CIFAR10	Average Points	61.9	56.0	62.1	60.5
	$\alpha < 0.95$	0/1024	0/1024	0/1024	0/1024
Random	Average Points	64.1	62.1	66.2	60.6
	$\alpha < 0.95$	1/800	0/800	0/800	0/800

C Related Work

The problem of extracting Deep Neural Networks (DNNs) traces back to the 1990s [1,2,3,12]. Early theoretical studies established that deducing the exact parameters of even a minimal neural network from a fixed, passively provided set of input-output examples is a formidable computational challenge, notoriously proven to be NP-complete [3]. However, the threat landscape shifts dramatically when attackers are empowered to adaptively choose their inputs. Driven by the widespread deployment of Machine Learning as a Service (MLaaS), this active black-box model extraction has resurged as a critical system security threat since Tramèr et al. [26] first demonstrated model stealing via predictive APIs in 2016. The core objective of modern extraction attacks [10,11,14,18,19,24] is either to infer the exact hidden weights and biases of neurons through limited chosen-input queries or to construct a functionally equivalent white-box substitute model.

Existing model extraction attacks primarily fall into two categories. The first category comprises *learning-based methods*, which aim to replicate the target model by training a substitute network on queried input-output pairs. However, these approaches suffer from an insurmountable bottleneck regarding extraction fidelity. Because deep learning optimization is inherently stochastic, the surrogate model invariably converges to a disparate local minimum, as profoundly highlighted by Jagielski et al. [14]. Consequently, learning-based methods can only ever achieve a rough “functional approximation.” Regardless of the query budget or optimization strategy, they fundamentally fail to achieve true parameter-level fidelity (i.e., exact mathematical recovery of the weights).

To breach this absolute fidelity barrier and transition from mere functional approximation to exact parameter equivalence, a second category emerged: *cryptanalytic methods*. Cryptanalytic methods aim to directly recover the exact underlying parameters by constructing and solving systems of algebraic equations, thereby achieving true mathematical fidelity. This paradigm was pioneered by Carlini et al. [6], who first translated the exact extraction of ReLU networks into the resolution of algebraic equations. By locating the non-differentiable boundaries of the activation function (i.e., where the pre-activation value strictly equals zero), they sequentially achieved signature and sign recovery for the neuron hyperplanes. This theoretical framework has since been significantly advanced: Canales-Martinez et al. [4] and Foerster et al. [11] successfully reduced the originally exponential sign recovery time down to polynomial time. Furthermore, recent studies have expanded the attack boundaries to highly restricted hard-label scenarios [5] and parameterized ReLU (PReLU) architectures [8].

Although cryptanalytic methods have achieved substantial progress in solving efficiency and adaptability to extreme environments, the academic community has not yet proposed a generic cryptanalytic scheme capable of handling smooth activations and expansive networks. This paper is explicitly motivated to fill this critical void in the field of security analysis.

It is worth noting that in the context of privacy-preserving neural network inference, MUSE[15] proposed a cryptanalytic parameter extraction attack executed by a malicious client. This approach requires the malicious client to ac-

tively break the cryptographic protocol by malleating the input secret shares. Consequently, the vulnerability of PPML systems under standard, protocol-compliant execution remains largely unexplored. Our work leverages the silent modular wraparound inherent in MPC to enable precise parameter extraction without deviating from the designated protocol.

D Proofs of Theorems

Theorem 1 Assume no wraparound points exist in the ϵ -neighborhood of x . Let the norm of each neuron's weight vector in layer i be bounded by K_i , i.e., $\|W_j^{(i)}\|_2 \leq K_i$ for all j , and let σ be a λ -Lipschitz continuous activation function. Then, for any perturbation with $\|\Delta x\|_2 < \epsilon$, the network output satisfies $\|f(x + \Delta x) - f(x)\|_2 \leq C\|\Delta x\|_2$, where $C = \lambda^r \prod_{i=1}^{r+1} \sqrt{d_i} K_i$.

Proof. Because there are no wraparound points in the ϵ -neighborhood of x , we can bound the output change for a single neuron j in the i -th linear layer as:

$$\begin{aligned} |z_j^{(i)}(x + \Delta x) - z_j^{(i)}(x)| &= |W_j^{(i)}(h^{(i-1)}(x + \Delta x) - h^{(i-1)}(x))| \\ &\leq \|W_j^{(i)}\|_2 \|h^{(i-1)}(x + \Delta x) - h^{(i-1)}(x)\|_2 \\ &\leq K_i \|h^{(i-1)}(x + \Delta x) - h^{(i-1)}(x)\|_2 \end{aligned}$$

For the entire vector of the i -th layer, the total change is:

$$\begin{aligned} \|z^{(i)}(x + \Delta x) - z^{(i)}(x)\|_2 &= \sqrt{\sum_{j=1}^{d_i} (z_j^{(i)}(x + \Delta x) - z_j^{(i)}(x))^2} \\ &\leq \sqrt{\sum_{j=1}^{d_i} (K_i \|h^{(i-1)}(x + \Delta x) - h^{(i-1)}(x)\|_2)^2} \\ &= \sqrt{d_i} K_i \|h^{(i-1)}(x + \Delta x) - h^{(i-1)}(x)\|_2 \end{aligned}$$

Because the activation function $\sigma(\cdot)$ is λ -Lipschitz continuous, the change after the activation layer is bounded by:

$$\|h^{(i)}(x + \Delta x) - h^{(i)}(x)\|_2 \leq \lambda \|z^{(i)}(x + \Delta x) - z^{(i)}(x)\|_2$$

The network f has $r + 1$ linear layers and r activation layers. By chaining these bounds from the input to the output, we get:

$$\begin{aligned} \|f(x + \Delta x) - f(x)\|_2 &\leq \left(\lambda^r \prod_{i=1}^{r+1} \sqrt{d_i} K_i \right) \|\Delta x\|_2 \\ &= C \|\Delta x\|_2 \end{aligned}$$

□

Theorem 2 Suppose x is a wraparound point in the i -th layer. Let $\mathcal{S}_\sigma = \{z \in \mathbb{R} \mid \exists \epsilon > 0, \forall y \in (z - \epsilon, z + \epsilon), \sigma(y) = \sigma(z)\}$ denote the flat region of the activation function, and let $p = \Pr[z \in \mathcal{S}_\sigma]$ be the probability that an intermediate pre-activation falls into this region. When traversing across this wraparound point, the probability that the network's final output remains continuous is bounded by $\Pr[\lim_{\|\Delta x\| \rightarrow 0} f(x + \Delta x) = f(x - \Delta x)] < \sum_{k=i+1}^r p^{d_k}$.

Proof. Assume x is a wraparound point in the i -th layer. When the input crosses this point, the output of the i -th layer has a sudden jump. Let this jump be:

$$\Delta h^{(i)} = \lim_{\|\Delta x\| \rightarrow 0} (h^{(i)}(x + \Delta x) - h^{(i)}(x - \Delta x)) \neq 0$$

Because the network weights are continuous, this sudden jump will not perfectly cancel out in the next layer. Therefore, the input to the $(i + 1)$ -th layer will also have a jump with a probability of 1:

$$\Pr \left(\lim_{\|\Delta x\| \rightarrow 0} W_j^{(i+1)} (h^{(i)}(x + \Delta x) - h^{(i)}(x - \Delta x)) \neq 0 \right) = 1$$

For the network to hide this jump (so the output becomes continuous again), the value before the activation function must fall into the flat region $\mathcal{S}_\sigma$. The chance of this happening for a single neuron is bounded by p :

$$\begin{aligned} \Pr \left(\lim_{\|\Delta x\| \rightarrow 0} h_j^{(i+1)}(x + \Delta x) = h_j^{(i+1)}(x - \Delta x) \right) &\leq \Pr \left(z_j^{(i+1)} \in \mathcal{S}_\sigma \right) \\ &= p \end{aligned}$$

For the entire $(i + 1)$ -th layer to hide the jump, all d_{i+1} neurons in this layer must independently fall into the flat region. The probability of this is:

$$\Pr \left(\lim_{\|\Delta x\| \rightarrow 0} h^{(i+1)}(x + \Delta x) = h^{(i+1)}(x - \Delta x) \right) \leq p^{d_{i+1}}$$

If the final output of the network is continuous, at least one hidden layer from $i + 1$ to r must completely hide the jump. Using the union bound, the total probability of a continuous final output is at most the sum of the probabilities for each layer:

$$\begin{aligned} &\Pr \left(\lim_{\|\Delta x\| \rightarrow 0} f(x + \Delta x) = f(x - \Delta x) \right) \\ &\leq \sum_{k=i+1}^r \Pr \left(\lim_{\|\Delta x\| \rightarrow 0} h^{(k)}(x + \Delta x) = h^{(k)}(x - \Delta x) \right) \\ &\leq \sum_{k=i+1}^r p^{d_k} \end{aligned}$$

□

Theorem 3 *The Jacobian matrix of the i -th layer can be factorized as $J^{(i)}(x) = \prod_{k=i}^1 (\Lambda^{(k)}(x) W^{(k)})$, where the diagonal activation derivative matrix is given by:*

$$\Lambda^{(k)}(x) = \text{diag} \left(\sigma'(z_1^{(k)}(x)), \sigma'(z_2^{(k)}(x)), \dots, \sigma'(z_{d_k}^{(k)}(x)) \right)$$

Furthermore, if x is a wraparound point of neuron $\eta_j^{(i)}$, its corresponding normal vector satisfies

$$n(x) = J^{(i-1)}(x)^T (W_j^{(i)})^T$$

Proof. First, we find the derivative of the k -th layer's output $h^{(k)}$ with respect to its input $h^{(k-1)}$ using the chain rule:

$$\frac{\partial h^{(k)}}{\partial h^{(k-1)}} = \frac{\partial h^{(k)}}{\partial z^{(k)}} \frac{\partial z^{(k)}}{\partial h^{(k-1)}}$$

Because $z^{(k)} = W^{(k)}h^{(k-1)} + b^{(k)}$, its derivative with respect to $h^{(k-1)}$ is simply the weight matrix $W^{(k)}$. The term $\frac{\partial h^{(k)}}{\partial z^{(k)}}$ is the derivative of the activation function. Because the activation function is applied element by element, this forms a diagonal matrix $\Lambda^{(k)}(x)$:

$$\frac{\partial h^{(k)}}{\partial z^{(k)}} = \Lambda^{(k)}(x) = \text{diag} \left(\sigma'(z_1^{(k)}(x)), \dots, \sigma'(z_{d_k}^{(k)}(x)) \right)$$

So, the derivative for a single layer is $\frac{\partial h^{(k)}}{\partial h^{(k-1)}} = \Lambda^{(k)}(x)W^{(k)}$. To find the full Jacobian matrix $J^{(i)}(x)$ from the network input x to the i -th layer, we multiply these single-layer matrices together:

$$J^{(i)}(x) = \frac{\partial h^{(i)}}{\partial x} = \prod_{k=i}^1 \frac{\partial h^{(k)}}{\partial h^{(k-1)}} = \prod_{k=i}^1 \left(\Lambda^{(k)}(x)W^{(k)} \right)$$

For the target neuron j in the i -th layer, its value before activation is $z_j^{(i)}(x) = W_j^{(i)}h^{(i-1)} + b_j^{(i)}$. Taking the gradient with respect to the input x gives the normal vector $n(x)$:

$$\begin{aligned} n(x) &= \nabla_x z_j^{(i)} \\ &= \left(\frac{\partial z_j^{(i)}}{\partial x} \right)^T \\ &= \left(\frac{\partial z_j^{(i)}}{\partial h^{(i-1)}} \frac{\partial h^{(i-1)}}{\partial x} \right)^T \\ &= \left(W_j^{(i)} J^{(i-1)}(x) \right)^T \\ &= J^{(i-1)}(x)^T (W_j^{(i)})^T \end{aligned}$$

□

Theorem 4 *Let σ be a non-polynomial analytic activation function, and let $x \in B(\eta_j^{(i)}, k)$ be a known wraparound point. Provided that the network's forward mapping F_{i-1} is non-degenerate within the local R -neighborhood of x , the affine span of the mapped local boundary strictly encompasses the entire signature hyperplane. That is, $\text{Aff}(B_{x,R}^{i-1}) = S(\eta_j^{(i)}, k)$.*

Proof. We use a proof by contradiction. Assume the mapped local boundary $\mathcal{M}$ does not span the entire signature hyperplane $S(\eta_j^{(i)}, k)$. This means $\mathcal{M}$ must lie inside a lower-dimensional subspace of $S(\eta_j^{(i)}, k)$. Therefore, there must exist a non-zero vector $v \in \mathbb{R}^{d_{i-1}}$ and a constant c such that:

1. v is parallel to the hyperplane, meaning it is orthogonal to the normal vector:
 $v \cdot W_j^{(i)} = 0$.
2. For all $x \in B(\eta_j^{(i)}, k) \cap \mathcal{N}(x^*, R)$, the equation $v \cdot F_{i-1}(x) = c$ holds.

By the definition of the wraparound boundary, for any point x in this neighborhood, we have:

$$\Phi(x) = \sum_{k=1}^{d_{i-1}} W_{j,k}^{(i)} F_{i-1,k}(x) - C = 0$$

At the same time, from our assumption, we have:

$$g(x) = \sum_{k=1}^{d_{i-1}} v_k F_{i-1,k}(x) - c = 0$$

Because the activation function is analytic, the identity theorem states that if $g(x) = 0$ wherever $\Phi(x) = 0$, they must be proportional. This means there is an analytic function $\lambda(x)$ such that for all x in the neighborhood:

$$\sum_{k=1}^{d_{i-1}} v_k F_{i-1,k}(x) - c = \lambda(x) \left(\sum_{k=1}^{d_{i-1}} W_{j,k}^{(i)} F_{i-1,k}(x) - C \right)$$

To make the proof easier to follow, consider the case where $i = 2$. The equation above becomes:

$$\begin{aligned} & \sum_{k=1}^{d_{i-1}} v_k \sigma(W_k^{(1)} x + b_k^{(1)}) - c \\ &= \lambda(x) \left(\sum_{k=1}^{d_{i-1}} W_{j,k}^{(i)} \sigma(W_k^{(1)} x + b_k^{(1)}) - C \right) \end{aligned}$$

If σ is not a polynomial, its ridge functions are linearly independent[16]. If $\lambda(x)$ is not a constant, its Taylor expansion has at least one non-zero term like $c_m x_m$. Multiplying this term with the right side creates mixed terms like $x_m \sigma(W_k^{(1)} x + b_k^{(1)})$.

However, the left side only contains simple ridge functions $\sigma(W_k^{(1)} x + b_k^{(1)})$. Because non-polynomial ridge functions are linearly independent, a simple ridge function can never equal a mixed term. To keep both sides strictly equal, the right side cannot have these mixed terms. This forces all non-constant terms in $\lambda(x)$ to be exactly 0.

Therefore, $\lambda(x)$ is just a constant λ . Matching the coefficients of the ridge functions on both sides, we get:

$$v_k = \lambda W_{j,k}^{(i)}, \quad \forall k \implies v = \lambda W_j^{(i)}$$

Substituting this into our first condition ($v \cdot W_j^{(i)} = 0$), we get:

$$(\lambda W_j^{(i)}) \cdot W_j^{(i)} = \lambda \|W_j^{(i)}\|_2^2 = 0$$

Because the weight vector $W_j^{(i)}$ is not zero ($\|W_j^{(i)}\|_2^2 > 0$), we must have $\lambda = 0$. This means $v = \mathbf{0}$, which directly contradicts our starting assumption that v is a non-zero vector.

This contradiction proves that our assumption is wrong. The mapped boundary must fully span the entire signature hyperplane $S(\eta_j^{(i)}, k)$. $\square$

Theorem 5 *Let $V_{\text{int}} = V_a \cap V_b$. Assume $\dim(V_{\text{int}}) \geq t$ and no two distinct neuron signature project collinearly onto any t -dimensional subspace. Then, for weights W_a and W_b :*

$$P_{\text{int}}((J_a^T)^\dagger n_a) \parallel P_{\text{int}}((J_b^T)^\dagger n_b) \iff W_a = W_b$$

where P_{int} is the orthogonal projection onto V_{int} , and $\parallel$ denotes vector collinearity.

Proof. First, we know the relationship between the normal vectors and the weights: $J_a^T W_a^T = n_a$ and $J_b^T W_b^T = n_b$.

The operation $(J_a^T)^\dagger n_a$ projects the weight vector W_a^T onto the row space of J_a^T . Because the row space of J_a^T is the visible subspace V_a , we can write:

$$(J_a^T)^\dagger n_a = (J_a^T)^\dagger J_a^T W_a^T = P_{V_a}(W_a^T)$$

where P_{V_a} is the projection onto V_a . Similarly, for the second point, we have $(J_b^T)^\dagger n_b = P_{V_b}(W_b^T)$.

Next, we apply the projection P_{int} onto the intersection space $V_{\text{int}} = V_a \cap V_b$. Because V_{int} is inside V_a ($V_{\text{int}} \subseteq V_a$), projecting a vector onto V_a and then onto V_{int} is the same as projecting it directly onto V_{int} . Using this property, we get:

$$\begin{aligned} P_{\text{int}}((J_a^T)^\dagger n_a) &= P_{\text{int}}(P_{V_a}(W_a^T)) \\ &= P_{\text{int}}(W_a^T) \end{aligned}$$

By the exact same logic, $P_{\text{int}}((J_b^T)^\dagger n_b) = P_{\text{int}}(W_b^T)$.

Now, we prove both directions of the theorem:

($\Leftarrow$): If $W_a = W_b$, their projections onto V_{int} are exactly the same: $P_{\text{int}}(W_a^T) = P_{\text{int}}(W_b^T)$. Because identical vectors are collinear, it directly follows that $P_{\text{int}}((J_a^T)^\dagger n_a) \parallel P_{\text{int}}((J_b^T)^\dagger n_b)$.

($\Rightarrow$): If $P_{\text{int}}((J_a^T)^\dagger n_a) \parallel P_{\text{int}}((J_b^T)^\dagger n_b)$, our derivation shows that $P_{\text{int}}(W_a^T) \parallel P_{\text{int}}(W_b^T)$. This means the projections of W_a and W_b are collinear in V_{int} . Because $\dim(V_{\text{int}}) \geq t$, they are collinear in a space of at least t dimensions. However, our assumption states that no two different neurons can have collinear projections in any t -dimensional space. Therefore, W_a and W_b must be the same neuron weight. Thus, $W_a = W_b$. $\square$

Theorem 6 Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = \alpha$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients satisfy $g_k^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2), g_k^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. Under these conditions, the success probability is

$$p = \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}} \right)$$

Proof. Because g_k^+ are independent zero-mean Gaussian variables, their linear combination y_v^+ is also a zero-mean Gaussian variable. Its variance is:

$$y_v^+ = \sum_{k=1}^{d_i} g_k^+ A_k v \sim \mathcal{N}(0, \sigma_{v+}^2), \quad \sigma_{v+}^2 = \sigma_+^2 \left((A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right)$$

Similarly, y_v^- is a zero-mean Gaussian variable with variance:

$$y_v^- = \sum_{k=1}^{d_i} g_k^- A_k v \sim \mathcal{N}(0, \sigma_{v-}^2), \quad \sigma_{v-}^2 = \sigma_-^2 \left(\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right)$$

Because y_v^+ and y_v^- are independent, the probability of a successful attack is the probability that $|y_v^+| > |y_v^-|$. We can write this as:

$$\begin{aligned} p &= \Pr(|y_v^+| > |y_v^-|) \\ &= \Pr\left(\frac{|y_v^+|}{|y_v^-|} > 1\right) \\ &= \Pr\left(\frac{|y_v^+|/\sigma_{v+}}{|y_v^-|/\sigma_{v-}} > \frac{\sigma_{v-}}{\sigma_{v+}}\right) \end{aligned}$$

The terms $|y_v^+|/\sigma_{v+}$ and $|y_v^-|/\sigma_{v-}$ are the absolute values of independent standard normal variables. Their ratio follows a standard half-Cauchy distribution, which has the cumulative distribution function $F(y) = \frac{2}{\pi} \arctan(y)$. Using this, the probability becomes:

$$p = 1 - \frac{2}{\pi} \arctan \left(\frac{\sigma_{v-}}{\sigma_{v+}} \right) = \frac{2}{\pi} \arctan \left(\frac{\sigma_{v+}}{\sigma_{v-}} \right)$$

Next, we can rewrite the ratio σ_{v+}/σ_{v-} using the signal-to-interference ratio, $\text{SIR}(v) = \frac{(A_j v)^2}{\sum_{k \neq j} (A_k v)^2}$:

$$\begin{aligned} \frac{\sigma_{v+}}{\sigma_{v-}} &= \sqrt{\frac{\sigma_+^2 \left((A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right)}{\sigma_-^2 \left(\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right)}} \\ &= \frac{\sigma_+}{\sigma_-} \sqrt{\frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}} \end{aligned}$$

Substituting this ratio back into the probability equation gives the final result:

$$p = \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}} \right)$$

□

Theorem 7 Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = 0$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients g_k^+ and g_k^- ($k \neq j$) are independent and identically distributed with finite variance σ_+^2 and σ_-^2 . Under the condition of asymptotic stability $\lim_{d_i \rightarrow \infty} \frac{\max_{k \neq j} (A_k v)^2}{\sum_{k \neq j} (A_k v)^2} = 0$, the asymptotic success probability is:

$$\lim_{d_i \rightarrow \infty} p = \Phi_2(\Gamma, \Gamma; \rho) + \Phi_2(-\Gamma, -\Gamma; \rho)$$

where Φ_2 denotes the standard bivariate normal cumulative distribution function, the integral boundary is $\Gamma = g_j^+ \sqrt{\frac{\text{SIR}(v)}{\sigma_+^2 + \sigma_-^2}}$, and the correlation coefficient is $\rho = \frac{\sigma_+^2 - \sigma_-^2}{\sigma_+^2 + \sigma_-^2}$. Furthermore, this asymptotic probability strictly monotonically increases with $\text{SIR}(v)$.

Proof. Because $\tau_j^+ = 1$, the positive derivative y_v^+ contains both the target signal and the background interference:

$$y_v^+ = g_j^+ A_j v + \sum_{k \neq j} g_k^+ A_k v$$

Because $\tau_j^- = 0$, the target neuron is inactive on the negative side. Therefore, the negative derivative y_v^- only contains the interference, and its signal term is exactly zero:

$$y_v^- = 0 + \sum_{k \neq j} g_k^- A_k v$$

By the Central Limit Theorem, as the layer width grows, the sums of the interference terms converge to normal distributions:

$$\begin{aligned} \sum_{k \neq j} g_k^+ A_k v &\xrightarrow{d} \mathcal{N}(0, \sigma_{v+}^2), \quad \sigma_{v+}^2 = \sigma_+^2 \sum_{k \neq j} (A_k v)^2 \\ \sum_{k \neq j} g_k^- A_k v &\xrightarrow{d} \mathcal{N}(0, \sigma_{v-}^2), \quad \sigma_{v-}^2 = \sigma_-^2 \sum_{k \neq j} (A_k v)^2 \end{aligned}$$

So, the full directional derivatives follow these distributions:

$$y_v^+ \xrightarrow{d} \mathcal{N}(g_j^+ A_j v, \sigma_{v+}^2), \quad y_v^- \xrightarrow{d} \mathcal{N}(0, \sigma_{v-}^2)$$

To easily compare their absolute values, we use the difference of squares:

$$|y_v^+| > |y_v^-| \iff (y_v^+ + y_v^-)(y_v^+ - y_v^-) > 0$$

Let $U = y_v^+ + y_v^-$ and $V = y_v^+ - y_v^-$. Their expected values are:

$$\mathbb{E}(U) = \mathbb{E}(V) = g_j^+ A_j v$$

Their variances and covariance are:

$$\begin{aligned}\text{Var}(U) &= \text{Var}(V) = \sigma_{v+}^2 + \sigma_{v-}^2 \\ \text{Cov}(U, V) &= \text{Var}(y_v^+) - \text{Var}(y_v^-) = \sigma_{v+}^2 - \sigma_{v-}^2\end{aligned}$$

Because U and V are linear combinations of independent Gaussian variables, they form a bivariate normal distribution. Their correlation coefficient ρ is:

$$\rho = \frac{\text{Cov}(U, V)}{\sqrt{\text{Var}(U)\text{Var}(V)}} = \frac{\sigma_{v+}^2 - \sigma_{v-}^2}{\sigma_{v+}^2 + \sigma_{v-}^2} = \frac{\sigma_+^2 - \sigma_-^2}{\sigma_+^2 + \sigma_-^2}$$

The probability of a successful attack is the chance that U and V share the same sign (both positive or both negative):

$$\Pr(|y_v^+| > |y_v^-|) = \Pr(U > 0, V > 0) + \Pr(U < 0, V < 0)$$

Using the standard bivariate normal cumulative distribution function Φ_2 , we divide the variables by their standard deviation $\sqrt{\sigma_{v+}^2 + \sigma_{v-}^2}$. Notice how the integration boundary Γ is formed using $\text{SIR}(v)$:

$$\begin{aligned}\frac{\mathbb{E}(U)}{\sqrt{\text{Var}(U)}} &= \frac{g_j^+ A_j v}{\sqrt{(\sigma_+^2 + \sigma_-^2) \sum_{k \neq j} (A_k v)^2}} \\ &= g_j^+ \sqrt{\frac{\text{SIR}(v)}{\sigma_+^2 + \sigma_-^2}} \\ &= \Gamma\end{aligned}$$

Using this boundary Γ , the asymptotic probability is:

$$\lim_{d_i \rightarrow \infty} \Pr(|y_v^+| > |y_v^-|) = \Phi_2(\Gamma, \Gamma; \rho) + \Phi_2(-\Gamma, -\Gamma; \rho)$$

To show this probability increases with $\text{SIR}(v)$, let $g(x) = \Phi_2(x, x; \rho) + \Phi_2(-x, -x; \rho)$. Its derivative is:

$$g'(x) = 2\phi(x) \left[\Phi\left(x \frac{\sigma_-}{\sigma_+}\right) - \Phi\left(-x \frac{\sigma_-}{\sigma_+}\right) \right]$$

Because $x > 0$ and the standard normal cumulative distribution $\Phi(z)$ is strictly increasing, we know $\Phi\left(x \frac{\sigma_-}{\sigma_+}\right) > \Phi\left(-x \frac{\sigma_-}{\sigma_+}\right)$. Also, the normal density $\phi(x)$ is always positive. Therefore, $g'(x) > 0$, meaning $g(x)$ is strictly increasing. Since the input Γ strictly increases with $\text{SIR}(v)$, the final success probability also strictly increases with $\text{SIR}(v)$. $\square$

Theorem 8 *The Signal-to-Interference Ratio $\text{SIR}(v) = \frac{(A_j v)^2}{\sum_{k \neq j} (A_k v)^2}$ is maximized in the direction $v = A^\dagger e_j$.*

Proof. We can write the signal-to-interference ratio $\text{SIR}(v)$ in matrix form:

$$\text{SIR}(v) = \frac{v^T A_j^T A_j v}{v^T A_{-j}^T A_{-j} v}$$

where A_{-j} is the matrix A without its j -th row. We want to find the vector v that maximizes this ratio.

Let $R = A_{-j}^T A_{-j}$. Assuming R is positive definite, it has an inverse R^{-1} and a square root $R^{1/2}$. We introduce a new variable $u = R^{1/2} v$, which means $v = R^{-1/2} u$. Substituting this into our ratio gives:

$$\max_u \frac{u^T R^{-1/2} A_j^T A_j R^{-1/2} u}{u^T u} = \max_u \frac{(A_j R^{-1/2} u)^2}{u^T u}$$

By the Cauchy-Schwarz inequality, the maximum is reached when u is proportional to the vector $R^{-1/2} A_j^T$. Changing back to v , the optimal direction is:

$$v_{\text{opt}} \propto R^{-1/2} (R^{-1/2} A_j^T) = R^{-1} A_j^T = (A_{-j}^T A_{-j})^{-1} A_j^T$$

Now, recall the definition of the pseudoinverse $A^\dagger = (A^T A)^{-1} A^T$. Multiplying by e_j gives the j -th column:

$$A^\dagger e_j = (A^T A)^{-1} A^T e_j = (A^T A)^{-1} A_j^T$$

We can split $A^T A$ into $A_{-j}^T A_{-j} + A_j^T A_j = R + A_j^T A_j$. Substituting this in:

$$A^\dagger e_j = (R + A_j^T A_j)^{-1} A_j^T$$

Using the Sherman-Morrison formula, we can invert this matrix:

$$(R + A_j^T A_j)^{-1} = R^{-1} - \frac{R^{-1} A_j^T A_j R^{-1}}{1 + A_j R^{-1} A_j^T}$$

Substituting this back into the expression for $A^\dagger e_j$, we get:

$$\begin{aligned} A^\dagger e_j &= \left(R^{-1} - \frac{R^{-1} A_j^T A_j R^{-1}}{1 + A_j R^{-1} A_j^T} \right) A_j^T \\ &= R^{-1} A_j^T - \frac{R^{-1} A_j^T (A_j R^{-1} A_j^T)}{1 + A_j R^{-1} A_j^T} \\ &= \left(1 - \frac{A_j R^{-1} A_j^T}{1 + A_j R^{-1} A_j^T} \right) R^{-1} A_j^T \\ &= \left(\frac{1}{1 + A_j R^{-1} A_j^T} \right) R^{-1} A_j^T \end{aligned}$$

Because $A_j R^{-1} A_j^T$ is a positive number, the fraction $\frac{1}{1+A_j R^{-1} A_j^T}$ is just a positive constant. This means $A^\dagger e_j$ is strictly proportional to $R^{-1} A_j^T$:

$$A^\dagger e_j \propto v_{opt}$$

Because $\text{SIR}(v)$ is a ratio, its value does not change if we scale the vector v . Therefore, choosing the direction $v = A^\dagger e_j$ will perfectly reach the maximum $\text{SIR}(v)$. $\square$

Theorem 9 Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = \alpha$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients satisfy $G_{l,k}^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2), G_{l,k}^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. Under these conditions, the success probability in top-1 score scenario is

$$p' = \frac{2}{\pi} \arctan \left(\frac{\sigma_+ p_{top+}^+}{\sigma_- p_{top-}^-} \sqrt{\frac{\sum_{l \neq top+} (p_l^+)^2}{\sum_{l \neq top-} (p_l^-)^2} \cdot \frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}} \right)$$

Proof. Using the derivative rule of the softmax function, the derivatives of the top probability p_{top} with respect to each output y_i are:

$$\frac{\partial p_{top}}{\partial y_{top}} = p_{top}(1 - p_{top}), \quad \frac{\partial p_{top}}{\partial y_l} = -p_{top} p_l \quad \forall l \neq top$$

Let p_v^+ be the directional derivative of the top probability on the positive side. By the chain rule, it can be written as:

$$\begin{aligned} p_v^+ &= \frac{\partial p_{top+}^+}{\partial v} = \frac{\partial p_{top+}^+}{\partial y} \frac{\partial f}{\partial v} \\ &= p_{top+}^+ (e_{top+} - p^+)^T G^+ (A^{(i)})^+ A v \end{aligned}$$

Because $\sum_{l=1}^n p_l^+ = 1$, we can simplify this derivative to:

$$p_v^+ = p_{top+}^+ \sum_{l \neq top+} p_l^+ \left(G_{top+}^+ (A^{(i)})^+ A v - G_l^+ (A^{(i)})^+ A v \right)$$

Because G^+ follows a zero-mean normal distribution, this linear combination p_v^+ is also a zero-mean normal variable. Its variance is:

$$p_v^+ \sim \mathcal{N}(0, \sigma_{v+}^2), \quad \sigma_{v+}^2 = 2(p_{top+}^+)^2 \sum_{l \neq top+} (p_l^+)^2 \sigma_+^2 \left((A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right)$$

Similarly, the directional derivative on the negative side, p_v^- is a zero-mean normal variable with variance:

$$\begin{aligned} p_v^- &\sim \mathcal{N}(0, \sigma_{v-}^2) \\ \sigma_{v-}^2 &= 2(p_{top-}^-)^2 \sum_{l \neq top-} (p_l^-)^2 \sigma_-^2 \left(\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right) \end{aligned}$$

Because p_v^+ and p_v^- are independent zero-mean normal variables, the ratio of their absolute values follows a Half-Cauchy distribution. We can find the success probability p' using the arctangent function:

$$p' = \Pr[|p_v^+| > |p_v^-|] = \frac{2}{\pi} \arctan\left(\frac{\sigma_{v+}}{\sigma_{v-}}\right)$$

We can rewrite the terms containing $(A_j v)^2$ using $\text{SIR}(v) = \frac{(A_j v)^2}{\sum_{k \neq j} (A_k v)^2}$:

$$\begin{aligned} \frac{\sigma_{v+}}{\sigma_{v-}} &= \frac{\sigma_{+p_{\text{top}+}}}{\sigma_{-p_{\text{top}-}}} \sqrt{\frac{\sum_{l \neq \text{top}+} (p_l^+)^2}{\sum_{l \neq \text{top}-} (p_l^-)^2}} \sqrt{\frac{(A_j v)^2 + \sum_{k \neq j} (A_k v)^2}{\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2}} \\ &= \frac{\sigma_{+p_{\text{top}+}}}{\sigma_{-p_{\text{top}-}}} \sqrt{\frac{\sum_{l \neq \text{top}+} (p_l^+)^2}{\sum_{l \neq \text{top}-} (p_l^-)^2}} \cdot \frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1} \end{aligned}$$

Substituting this ratio back into the probability equation gives the final result:

$$p' = \frac{2}{\pi} \arctan\left(\frac{\sigma_{+p_{\text{top}+}}}{\sigma_{-p_{\text{top}-}}} \sqrt{\frac{\sum_{l \neq \text{top}+} (p_l^+)^2}{\sum_{l \neq \text{top}-} (p_l^-)^2}} \cdot \frac{\text{SIR}(v) + 1}{\alpha^2 \text{SIR}(v) + 1}\right)$$

□

Theorem 10 Suppose the activation states on both sides of the wraparound boundary are $\tau_j^+ = 1, \tau_j^- = \alpha$, and $\tau_k = 1, \forall k \neq j$. Additionally, assume the corresponding deep coefficients satisfy $g_k^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2), g_k^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. Under these conditions, the success probability based on the noise direction is

$$p'' = 1 - \frac{4}{\pi^2} \int_0^\infty \frac{\arctan(kt)}{t^2 + 1} dt$$

where the simplification constant is $k = \sqrt{\frac{\alpha^2 \text{SIR}(v) + 1}{\text{SIR}(v) + 1}}$.

Proof. From our previous derivations, the directional derivatives along the probe direction v follow the normal distributions:

$$\begin{aligned} y_v^+ &\sim \mathcal{N}(0, \sigma_{v+}^2), \quad \sigma_{v+}^2 = \sigma_+^2 \left((A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right) \\ y_v^- &\sim \mathcal{N}(0, \sigma_{v-}^2), \quad \sigma_{v-}^2 = \sigma_-^2 \left(\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2 \right) \end{aligned}$$

For the noise direction u , which is designed not to excite the target neuron ($A_j u = 0$), the distributions are:

$$y_u^+ \sim \mathcal{N}(0, \sigma_{u+}^2), \quad \sigma_{u+}^2 = \sigma_+^2 \sum_{k \neq j} (A_k u)^2$$

$$y_u^- \sim \mathcal{N}(0, \sigma_{u-}^2), \quad \sigma_{u-}^2 = \sigma_-^2 \sum_{k \neq j} (A_k u)^2$$

We define the ratios $\tau^+ = \frac{|y_v^+|}{|y_u^+|}$ and $\tau^- = \frac{|y_v^-|}{|y_u^-|}$. Because the unbalanced variances (σ_+^2 and σ_-^2) cancel out during the division on each side, τ^+ and τ^- strictly follow Half-Cauchy distributions:

$$\tau^+ \sim \text{Half-Cauchy}(0, \gamma_+), \quad \tau^- \sim \text{Half-Cauchy}(0, \gamma_-)$$

where the scaling parameters γ_+ and γ_- are:

$$\gamma_+ = \frac{\sigma_{v+}}{\sigma_{u+}} = \sqrt{\frac{(A_j v)^2 + \sum_{k \neq j} (A_k v)^2}{\sum_{k \neq j} (A_k u)^2}}$$

$$\gamma_- = \frac{\sigma_{v-}}{\sigma_{u-}} = \sqrt{\frac{\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2}{\sum_{k \neq j} (A_k u)^2}}$$

Let $X = \tau^+$ and $Y = \tau^-$. The probability density function (PDF) of Y and the cumulative distribution function (CDF) of X are:

$$f_Y(y) = \frac{2\gamma_-}{\pi(y^2 + \gamma_-^2)}, \quad F_X(x) = \frac{2}{\pi} \arctan\left(\frac{x}{\gamma_+}\right)$$

Because the measurements on the positive and negative sides are independent, X and Y are independent random variables. The success probability p'' is:

$$\begin{aligned} p'' &= \Pr(X > Y) \\ &= \int_0^\infty f_Y(y) \Pr(X > y) dy \\ &= \int_0^\infty f_Y(y) [1 - F_X(y)] dy \end{aligned}$$

Substituting the PDF and CDF into the integral gives:

$$p'' = \int_0^\infty \frac{2\gamma_-}{\pi(y^2 + \gamma_-^2)} \left[1 - \frac{2}{\pi} \arctan\left(\frac{y}{\gamma_+}\right) \right] dy$$

We define a constant k using the signal-to-interference ratio $\text{SIR}(v)$:

$$\begin{aligned} k &= \frac{\gamma_-}{\gamma_+} \\ &= \sqrt{\frac{\alpha^2 (A_j v)^2 + \sum_{k \neq j} (A_k v)^2}{(A_j v)^2 + \sum_{k \neq j} (A_k v)^2}} \\ &= \sqrt{\frac{\alpha^2 \text{SIR}(v) + 1}{\text{SIR}(v) + 1}} \end{aligned}$$

By substituting $t = \frac{y}{\gamma_-}$, we have $dy = \gamma_- dt$ and $\frac{y}{\gamma_+} = \frac{\gamma_- t}{\gamma_+} = kt$. This simplifies the integral to the final expression:

$$\begin{aligned} p'' &= \int_0^\infty \frac{2}{\pi(t^2 + 1)} \left[1 - \frac{2}{\pi} \arctan(kt) \right] dt \\ &= 1 - \frac{4}{\pi^2} \int_0^\infty \frac{\arctan(kt)}{t^2 + 1} dt \end{aligned}$$

□

Theorem 11 Assume the target neuron's activation states across the boundary are $\tau_j^+ = 1$ and $\tau_j^- = \alpha$. Also, assume the deep coefficients satisfy $g_k^+ \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_+^2)$ and $g_k^- \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma_-^2)$. If the network satisfies the following stability condition as it grows larger

$$\lim_{d_i \rightarrow \infty} \frac{\sum_{k \neq j} (A_k v)^4 \text{Var}(\tau_k^2)}{\left(\sum_{k \neq j} (A_k v)^2 \mathbb{E}[\tau_k^2] \right)^2} = 0$$

Then the asymptotic success probability is

$$\lim_{d_i \rightarrow \infty} p = \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{(A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2}{\alpha^2 (A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2}} \right)$$

Furthermore, this success probability reaches its maximum value when we use the direction $v = (A')^\dagger e_j$, where $A'_j = A_j$, and $A'_k = \sqrt{w_k} A_k$ ($\forall k \neq j$).

Proof. First, for a specific set of activation states $\boldsymbol{\tau}$, we can write the directional derivatives as:

$$y_v^+ = g_j^+ A_j v + \sum_{k \neq j} g_k^+ \tau_k A_k v, \quad y_v^- = g_j^- \alpha A_j v + \sum_{k \neq j} g_k^- \tau_k A_k v$$

Because g_k^+ and g_k^- are independent zero-mean Gaussian variables, y_v^+ and y_v^- are also independent Gaussian variables. Their conditional variances are:

$$\begin{aligned} \text{Var}(y_v^+ | \boldsymbol{\tau}) &= \sigma_+^2 \left((A_j v)^2 + \sum_{k \neq j} \tau_k^2 (A_k v)^2 \right) \\ \text{Var}(y_v^- | \boldsymbol{\tau}) &= \sigma_-^2 \left(\alpha^2 (A_j v)^2 + \sum_{k \neq j} \tau_k^2 (A_k v)^2 \right) \end{aligned}$$

Let $S = \sum_{k \neq j} \tau_k^2 (A_k v)^2$ be the sum of the background interference. Its expected value is:

$$\mathbb{E}[S] = \sum_{k \neq j} \mathbb{E}[\tau_k^2] (A_k v)^2 = \sum_{k \neq j} w_k (A_k v)^2$$

The stability condition in the theorem guarantees that:

$$\lim_{d_i \rightarrow \infty} \frac{\text{Var}(S)}{(\mathbb{E}[S])^2} = \lim_{d_i \rightarrow \infty} \frac{\sum_{k \neq j} (A_k v)^4 \text{Var}(\tau_k^2)}{\left(\sum_{k \neq j} (A_k v)^2 \mathbb{E}[\tau_k^2] \right)^2} = 0$$

By the Law of Large Numbers, this means that as the network width grows ($d_i \rightarrow \infty$), the random variable S converges to its exact expected value $\mathbb{E}[S]$. Therefore, for large networks, the variances become fixed values, which we denote as σ_{v+}^2 and σ_{v-}^2 :

$$\sigma_{v+}^2 = \sigma_+^2 \left((A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2 \right), \quad \sigma_{v-}^2 = \sigma_-^2 \left(\alpha^2 (A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2 \right)$$

Because the measurements are independent, the ratio $|y_v^+|/|y_v^-|$ follows a Half-Cauchy distribution. As shown in our earlier derivations, the success probability $\Pr(|y_v^+| > |y_v^-|)$ can be calculated using the arctangent function:

$$\begin{aligned} \lim_{d_i \rightarrow \infty} p &= \Pr(|y_v^+| > |y_v^-|) \\ &= \frac{2}{\pi} \arctan \left(\frac{\sigma_{v+}}{\sigma_{v-}} \right) \\ &= \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{(A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2}{\alpha^2 (A_j v)^2 + \sum_{k \neq j} w_k (A_k v)^2}} \right) \end{aligned}$$

We can define a refined signal-to-interference ratio using the modified matrix A' , where $A'_j = A_j$ and $A'_k = \sqrt{w_k} A_k$ for all $k \neq j$:

$$\text{SIR}_{\text{refined}}(v) = \frac{(A'_j v)^2}{\sum_{k \neq j} (A'_k v)^2}$$

Substituting this into our probability formula gives:

$$\lim_{d_i \rightarrow \infty} p = \frac{2}{\pi} \arctan \left(\frac{\sigma_+}{\sigma_-} \sqrt{\frac{\text{SIR}_{\text{refined}}(v) + 1}{\alpha^2 \text{SIR}_{\text{refined}}(v) + 1}} \right)$$

The optimal direction $v = (A')^\dagger e_j$ maximizes this probability, which (as in the standard scenario) strictly increases with $\text{SIR}_{\text{refined}}(v)$. $\square$

E Algorithm

Algorithm 5 Wraparound Point Search Algorithm

Input: Model oracle $\mathcal{O}$, step size s , jump threshold $\mathcal{M}$, tolerance ϵ

Output: Wraparound point x^*

```

1:  $x_0 \xleftarrow{\$} \mathbb{R}^{d_0}$ ,  $v \xleftarrow{\$} \{v \in \mathbb{R}^{d_0} \mid \|v\|_2 = s\}$ ,  $n \leftarrow 0$ 
2: loop
3:    $y_n \leftarrow \mathcal{O}(x_0 + nv)$ 
4:    $y_{n+1} \leftarrow \mathcal{O}(x_0 + (n+1)v)$ 
5:   if  $\|y_{n+1} - y_n\|_2 > \mathcal{M}$  then
6:      $\alpha_L \leftarrow n$ ,  $\alpha_H \leftarrow n+1$ 
7:     while  $\alpha_H - \alpha_L \geq \epsilon$  do
8:        $\alpha_M \leftarrow (\alpha_H + \alpha_L)/2$ 
9:        $\Delta_H \leftarrow \|\mathcal{O}(x_0 + \alpha_H v) - \mathcal{O}(x_0 + \alpha_M v)\|_2$ 
10:       $\Delta_L \leftarrow \|\mathcal{O}(x_0 + \alpha_M v) - \mathcal{O}(x_0 + \alpha_L v)\|_2$ 
11:      if  $\Delta_H > \mathcal{M}$  then
12:         $\alpha_L \leftarrow \alpha_M$ 
13:      else if  $\Delta_L > \mathcal{M}$  then
14:         $\alpha_H \leftarrow \alpha_M$ 
15:      else
16:        break
17:      if  $\alpha_H - \alpha_L < \epsilon$  then
18:        return  $x^* \leftarrow x_0 + \alpha_M v$ 
19:     $n \leftarrow n+1$ 

```

Algorithm 6 Normal Vector Calculation Algorithm

Input: Wraparound point x , step size α , orthogonal basis $\{e_i\}_{i=0}^{d_0-1}$

Output: Normal vector $\mathbf{n}$

```

1:  $n_0 \leftarrow 1$ 
2:  $x' \leftarrow x + \alpha e_0$ 
3: for  $i = 1$  to  $d_0 - 1$  do
4:    $\beta_i \leftarrow \arg \min_{\beta} \{|\beta| \mid x' + \beta e_i \in \mathcal{W}\}$  % via binary search
5:    $n_i \leftarrow -\beta_i/\alpha$ 
6:  $\mathbf{n}_{\text{raw}} \leftarrow (n_0, n_1, \dots, n_{d_0-1})^T$ 
7: return  $\mathbf{n} \leftarrow \mathbf{n}_{\text{raw}}/\|\mathbf{n}_{\text{raw}}\|_2$ 

```
